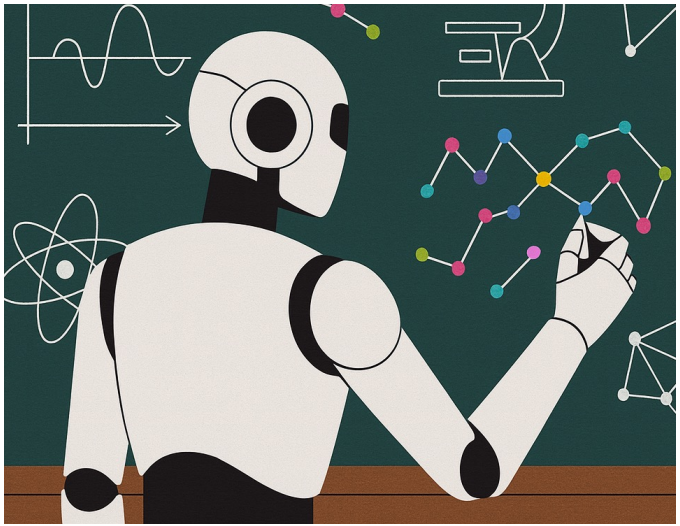
AI for (Quantum) Physics

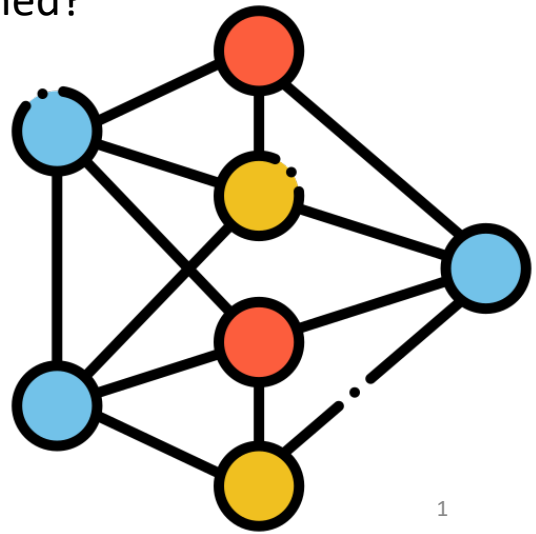


Xuemei Gu
xuemei.gu@uni-jena.de

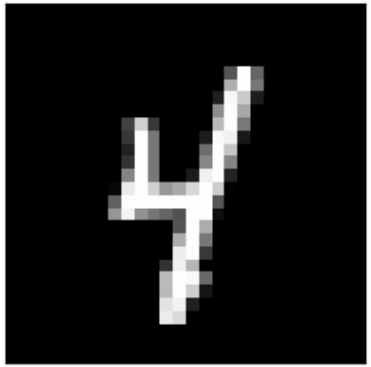
If you have any questions, feel free to reach out!

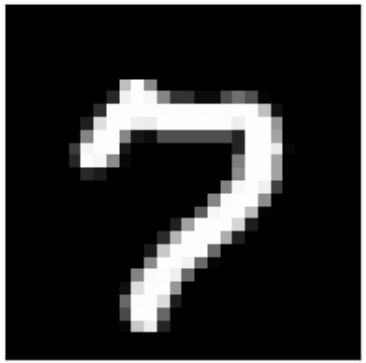
Today's topic:

- What is a neural network?
- How is it trained?
- Examples
- Discussion

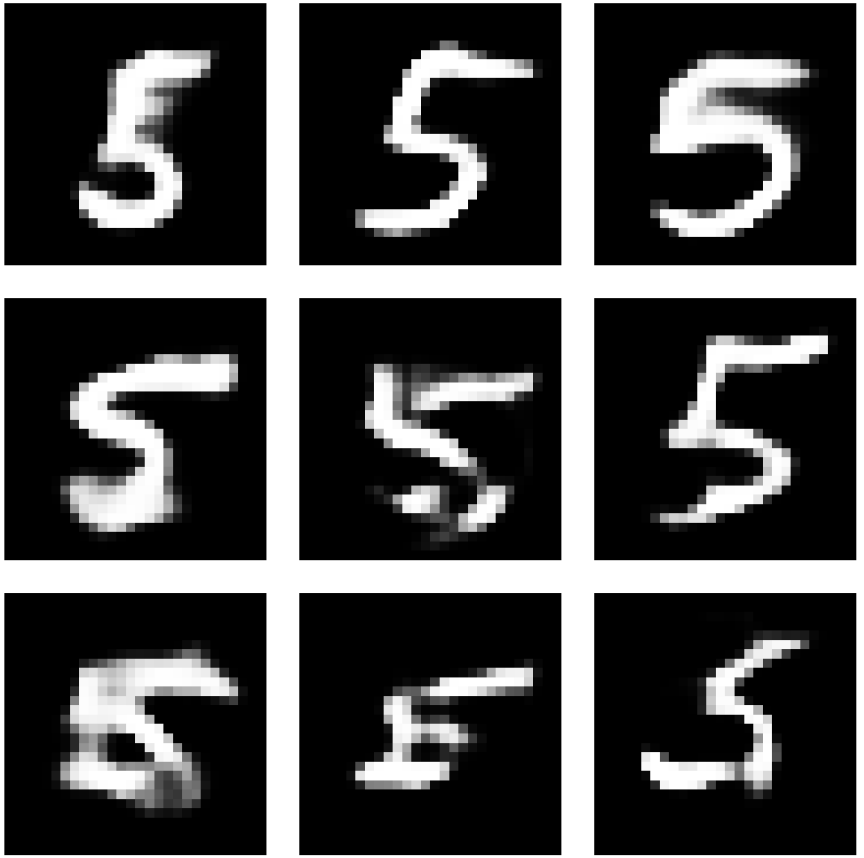


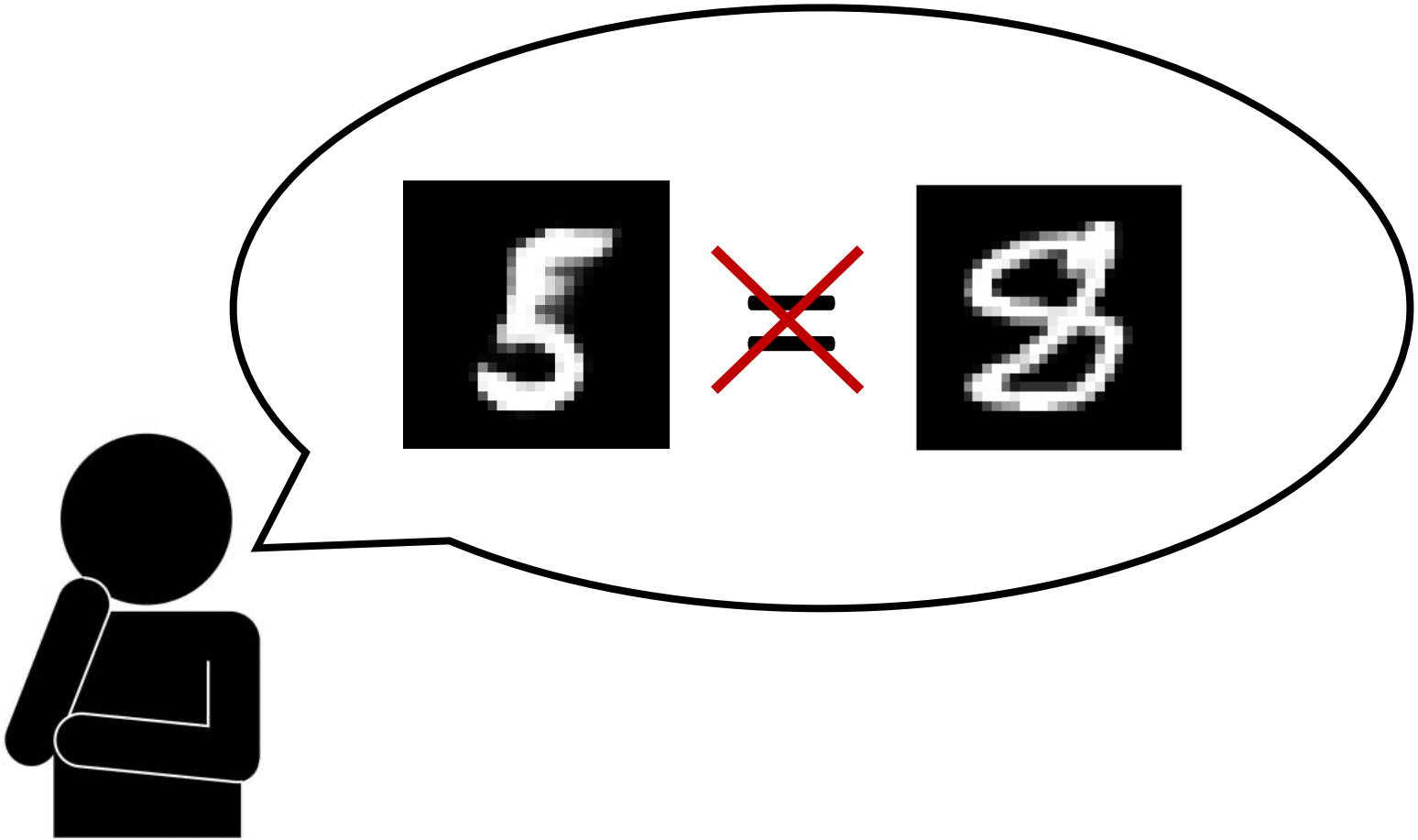


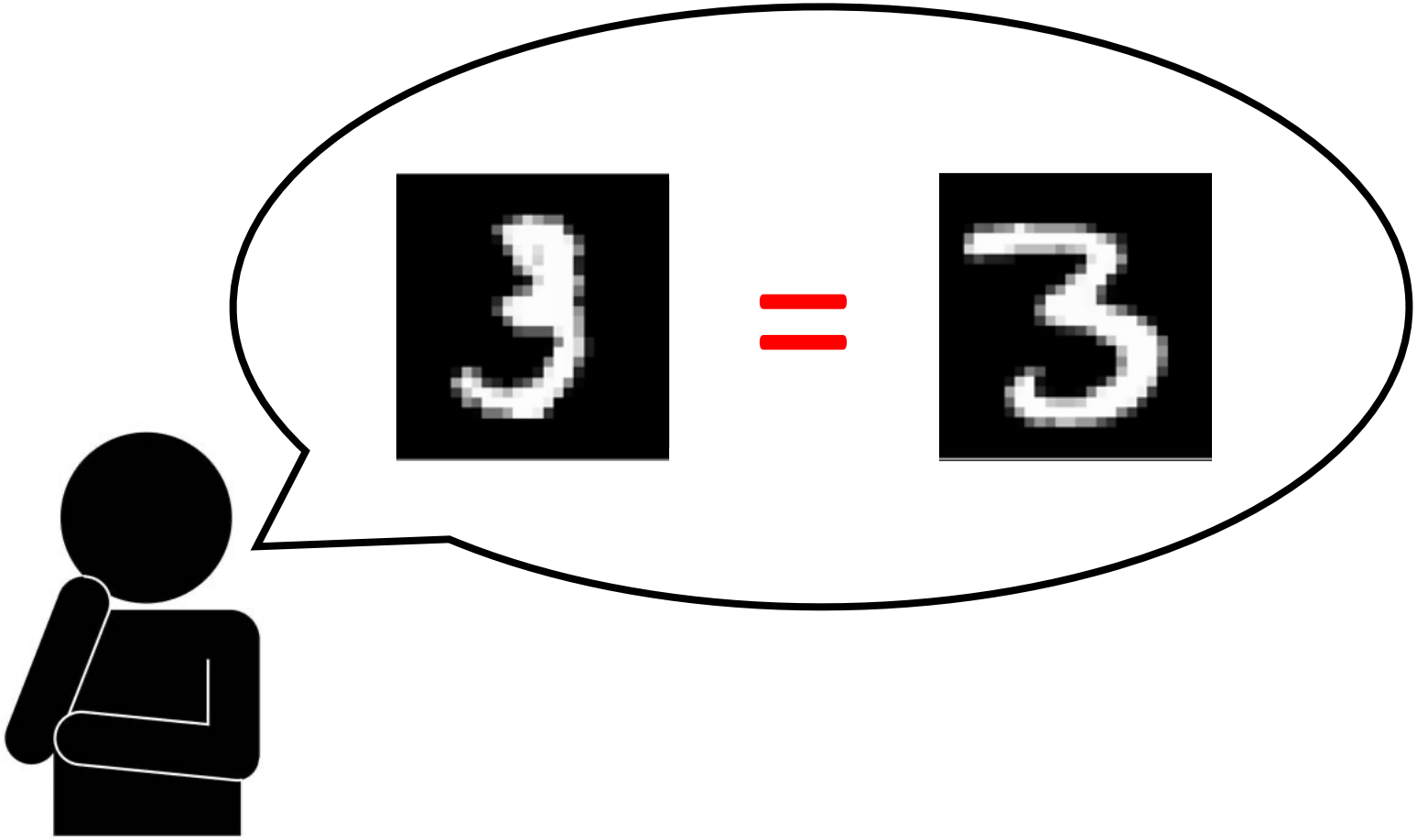


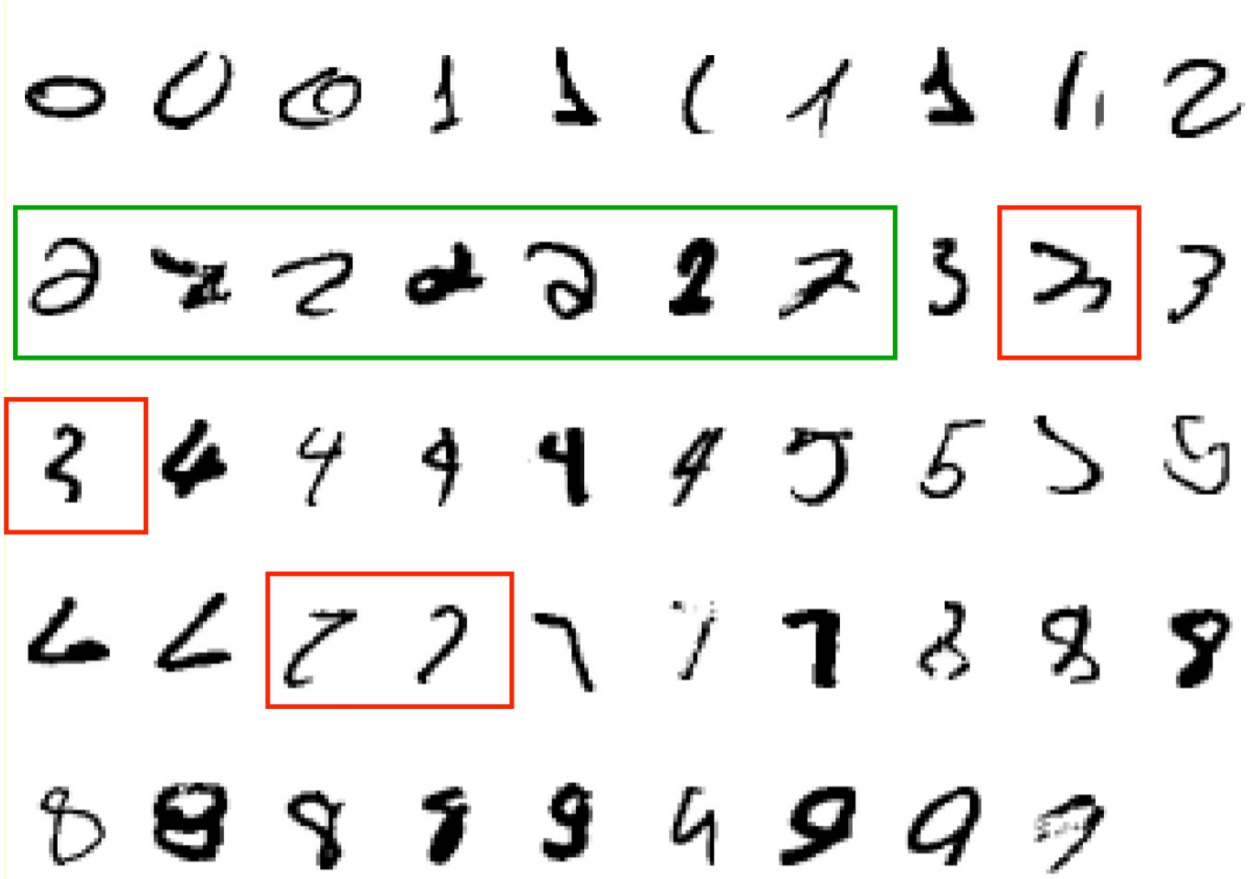






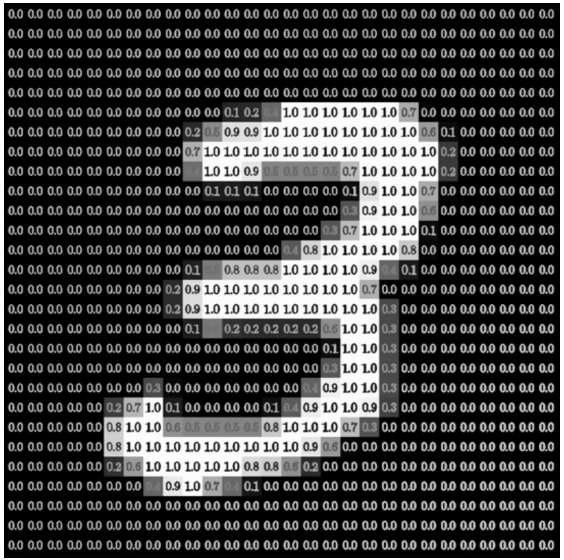


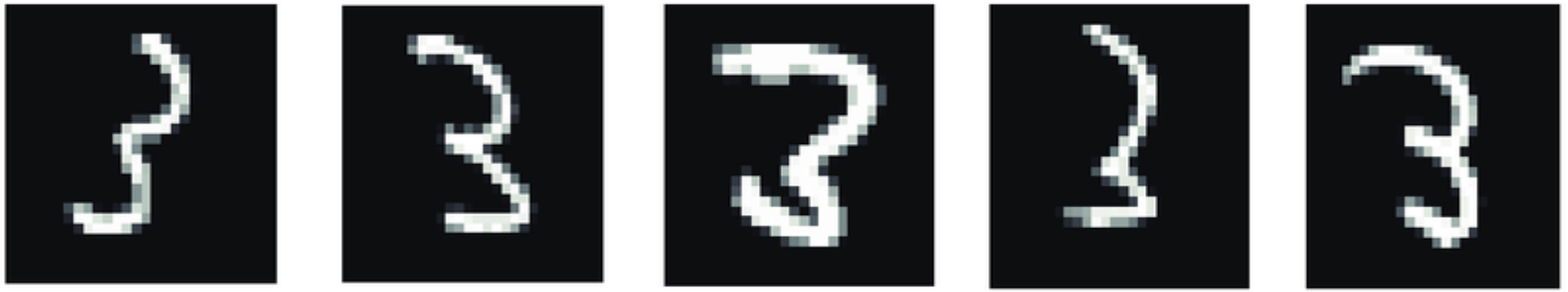




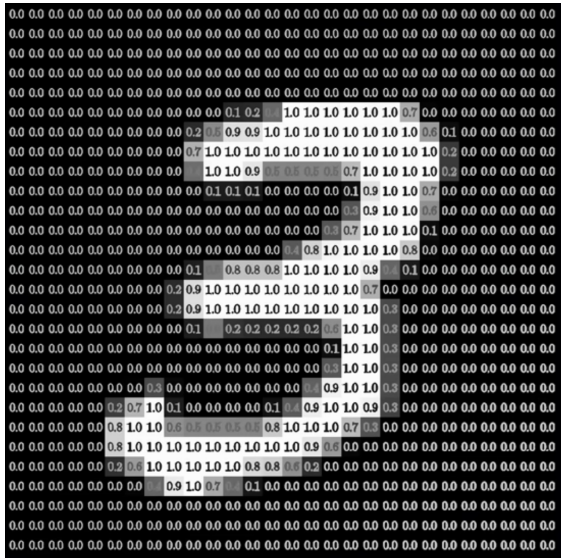
From geoffrey hinton’s lecture

MNIST database handwritten digits





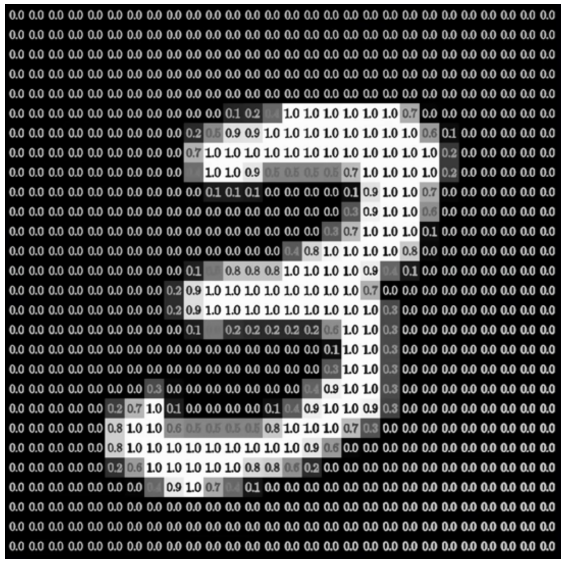
How can we ask a program to recognize this?





0	1	2	3	4	5	6	7	8	9
?	?	?	?	?	?	?	?	?	?

How can we ask a program to recognize this?



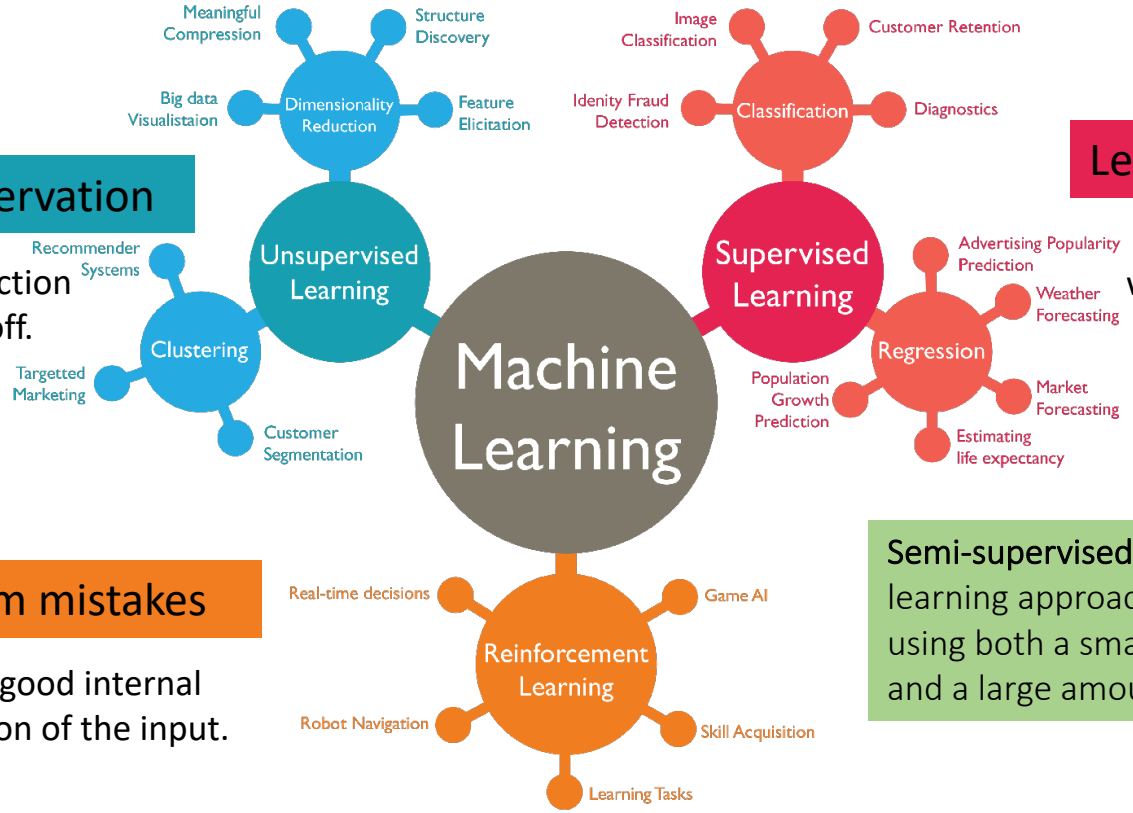
Teach computer to tell the number



3

What is Machine Learning?

Machine learning (ML) a field of artificial intelligence that uses algorithms to learn from data, identify patterns, and make predictions or decisions without being explicitly programmed.



Learning by observation

Learn to select an action to maximize payoff.

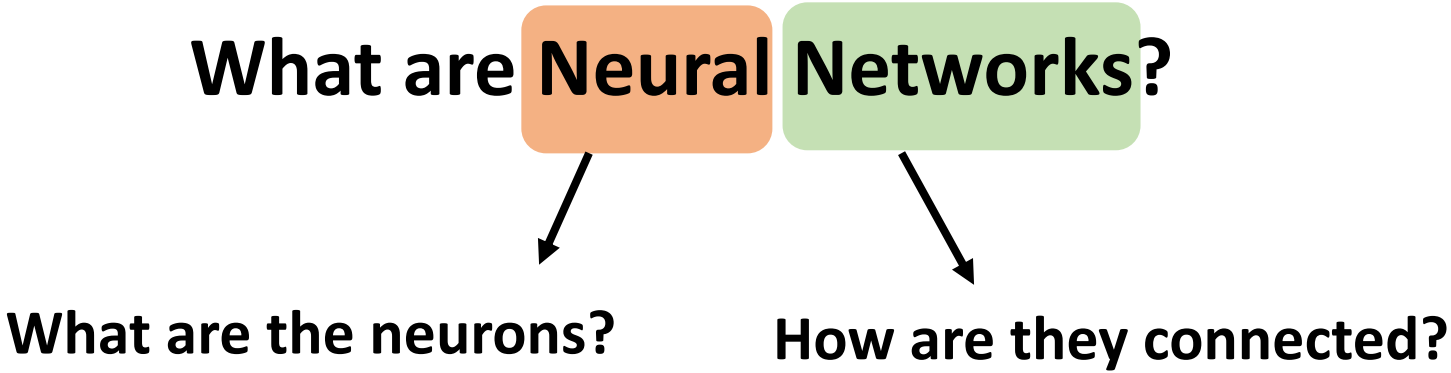
Learning by examples

Learn to predict an output when given an input vector.

Learning from mistakes

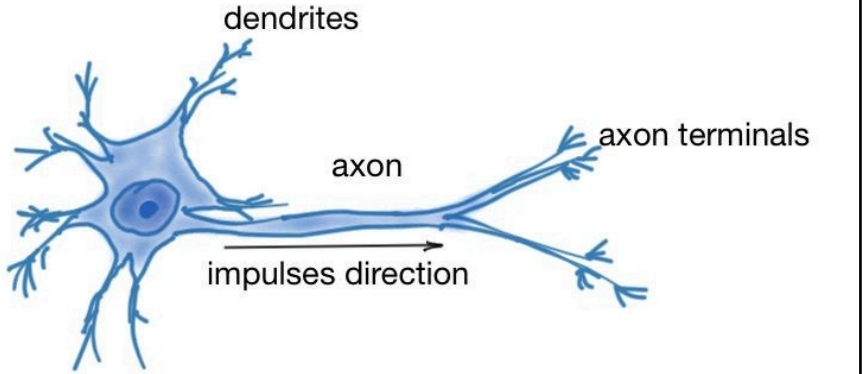
Discover a good internal representation of the input.

Semi-supervised learning is a machine learning approach that trains AI models using both a small amount of labelled data and a large amount of unlabelled data.

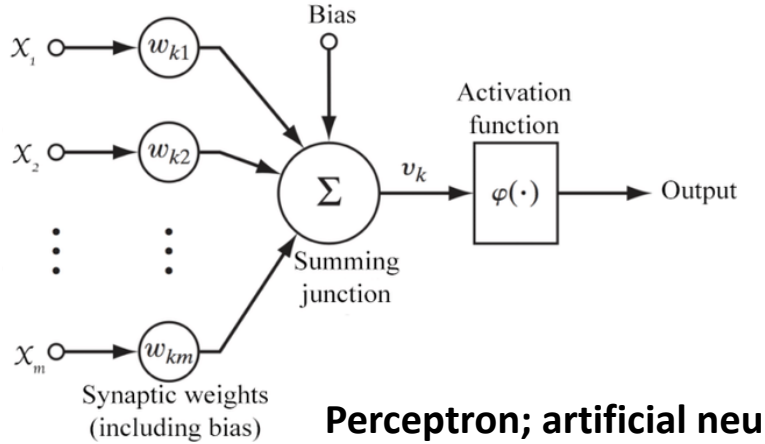

What are Neural Networks?
What are the neurons?
How are they connected?

What are Neural Networks?

Biological Inspiration; brain-inspired model



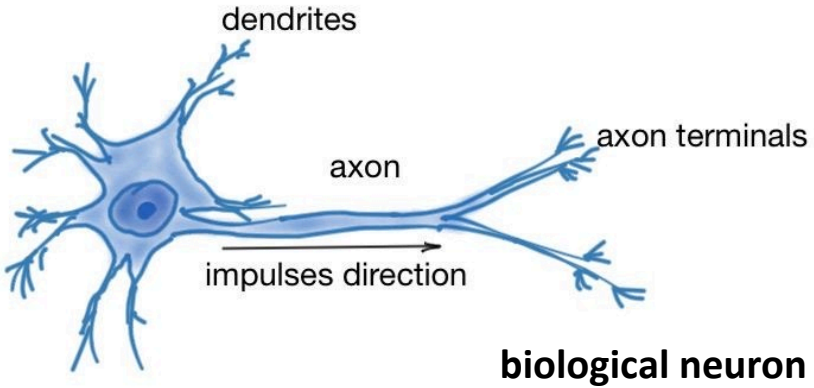
biological neuron



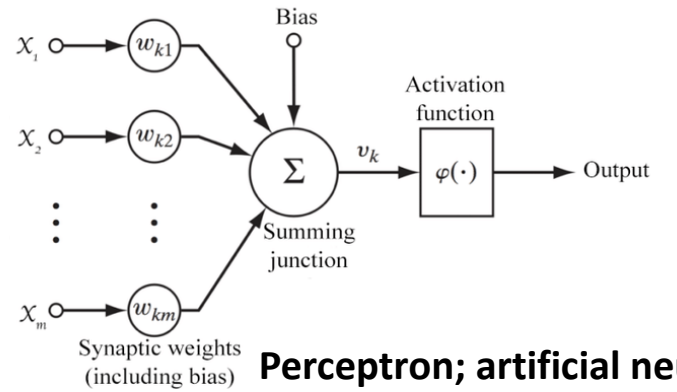
Perceptron; artificial neuron

- Axons typically contact dendritic trees at synapses
  - A spike of activity in the axon causes charge to be injected into the post-synaptic neuron.
- Each neuron receives inputs from other neurons
- The effect of each input line on the neuron is controlled by a synaptic weight (positive or negative).
- The synaptic weights adapt so that the whole network learns to perform useful computations.

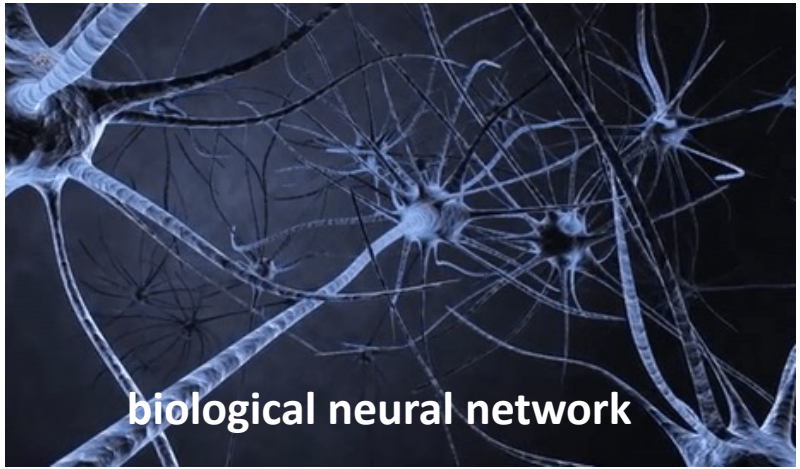
What are Neural Networks?



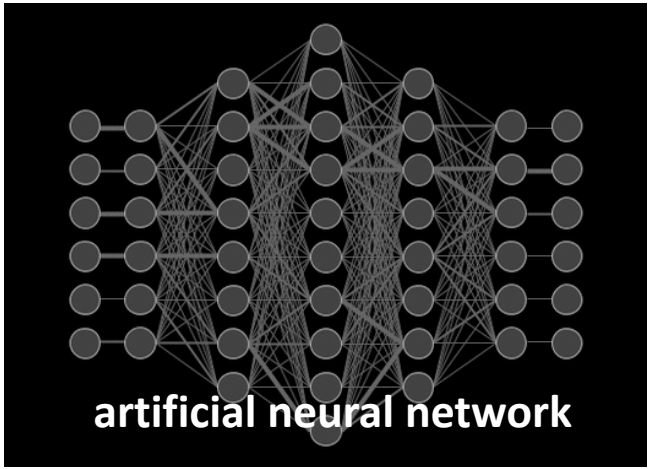
biological neuron



Perceptron; artificial neuron



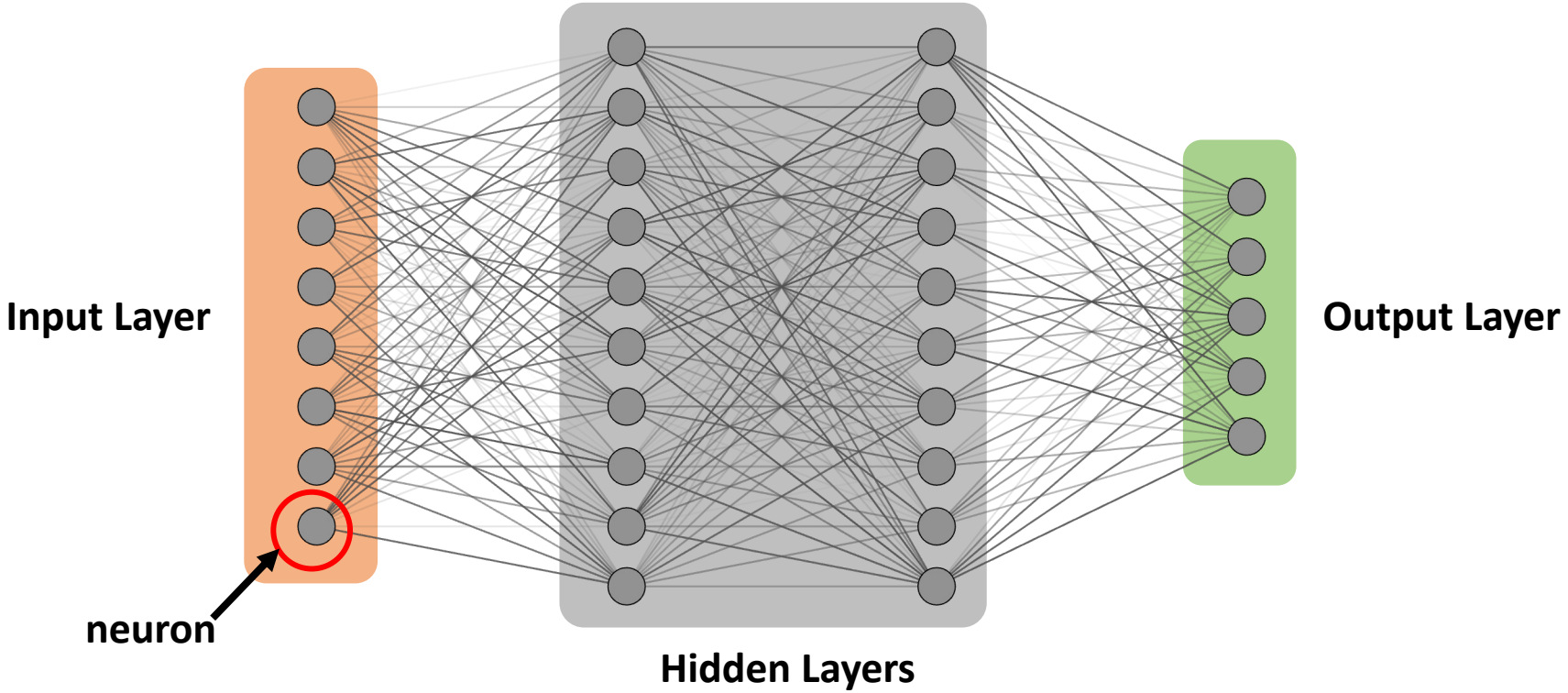
biological neural network



artificial neural network

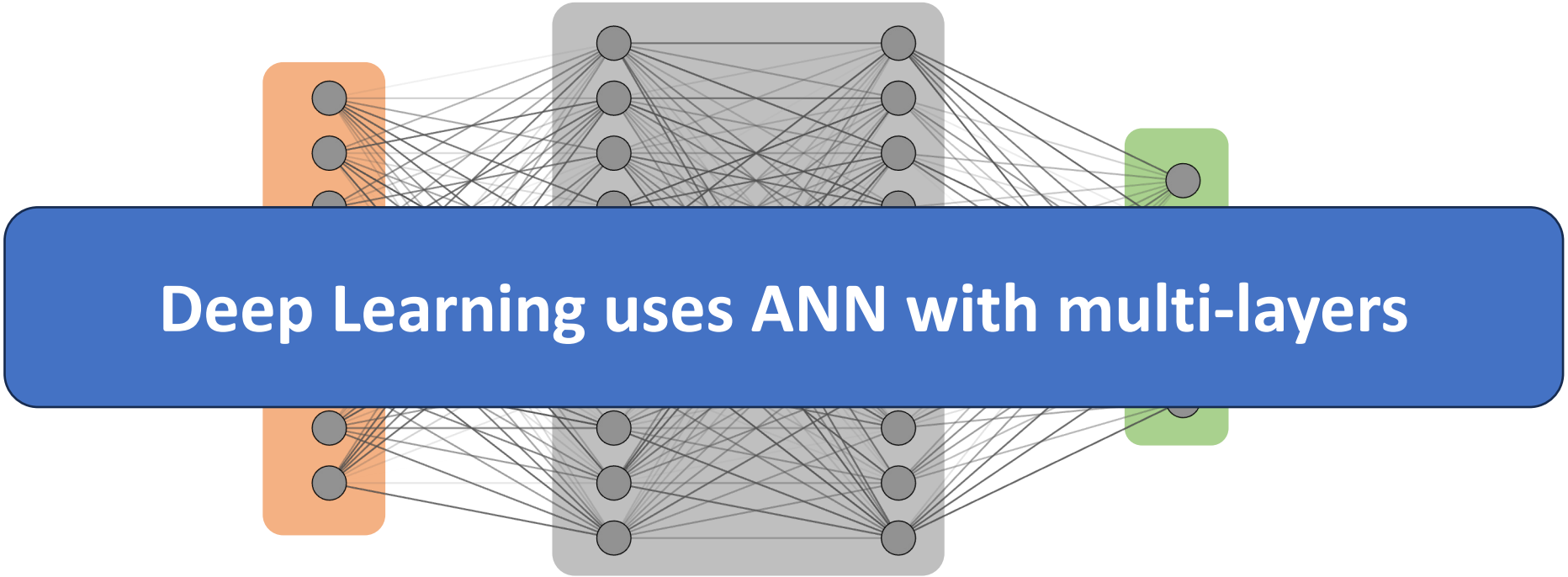
What are Neural Networks?

In machine learning, a neural network, also called **artificial neural network**(**ANN**), is a computational model inspired by the structure and functions of biological neural networks



What are Neural Networks?

In machine learning, a neural network, also called **artificial neural network**(**ANN**), is a computational model inspired by the structure and functions of biological neural networks



How does Neural Network learn?

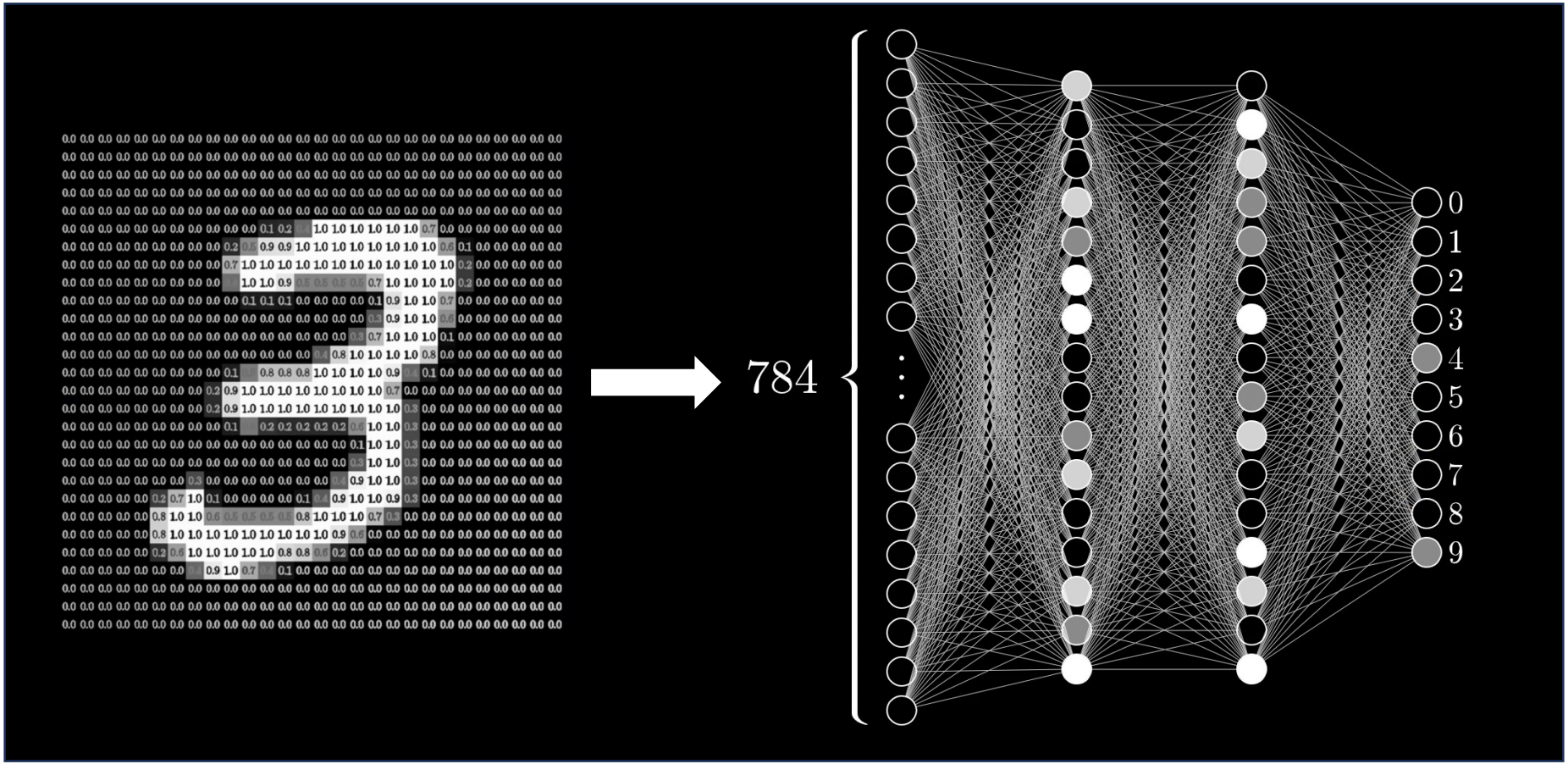
28x28=784 pixel

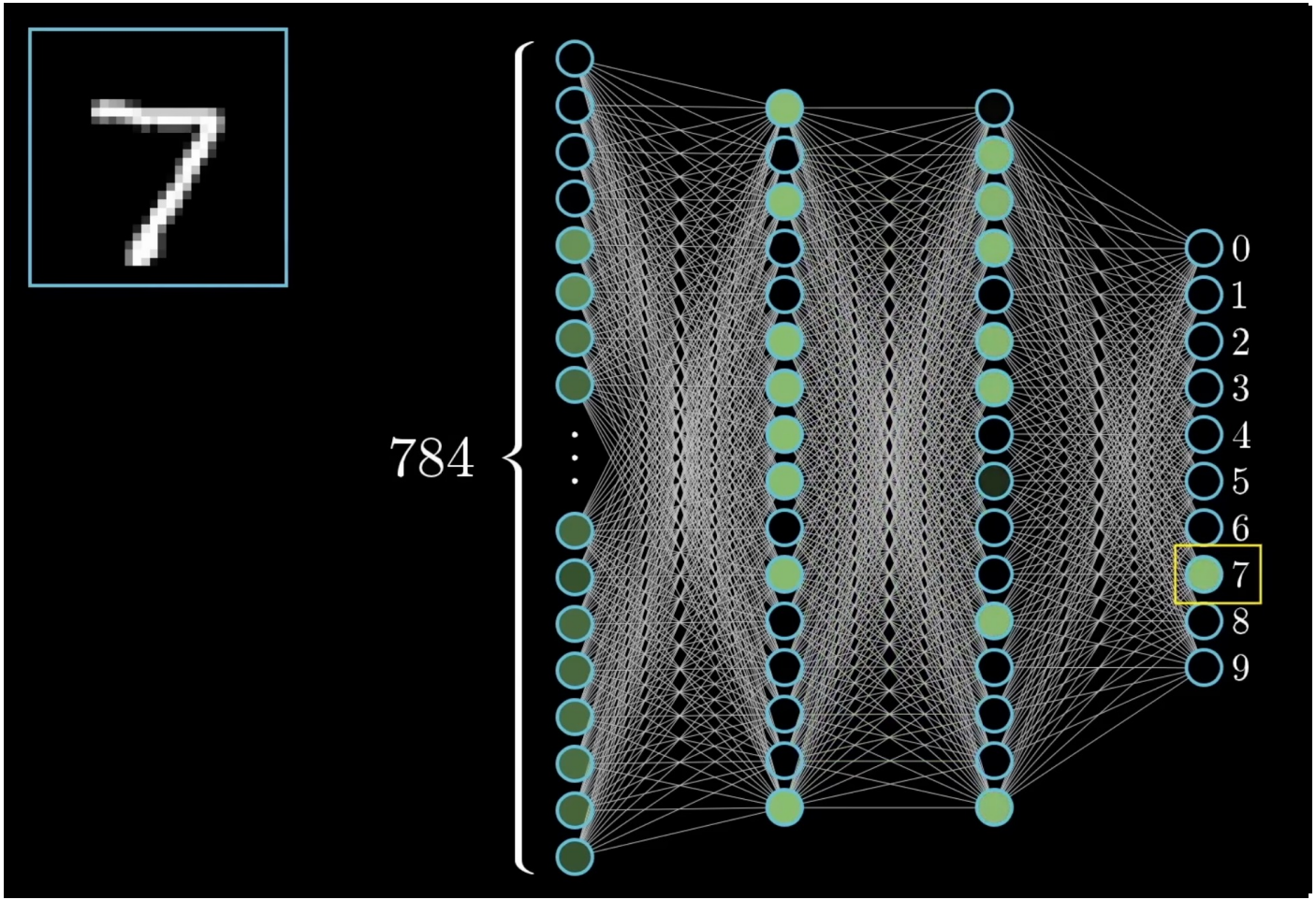
activation

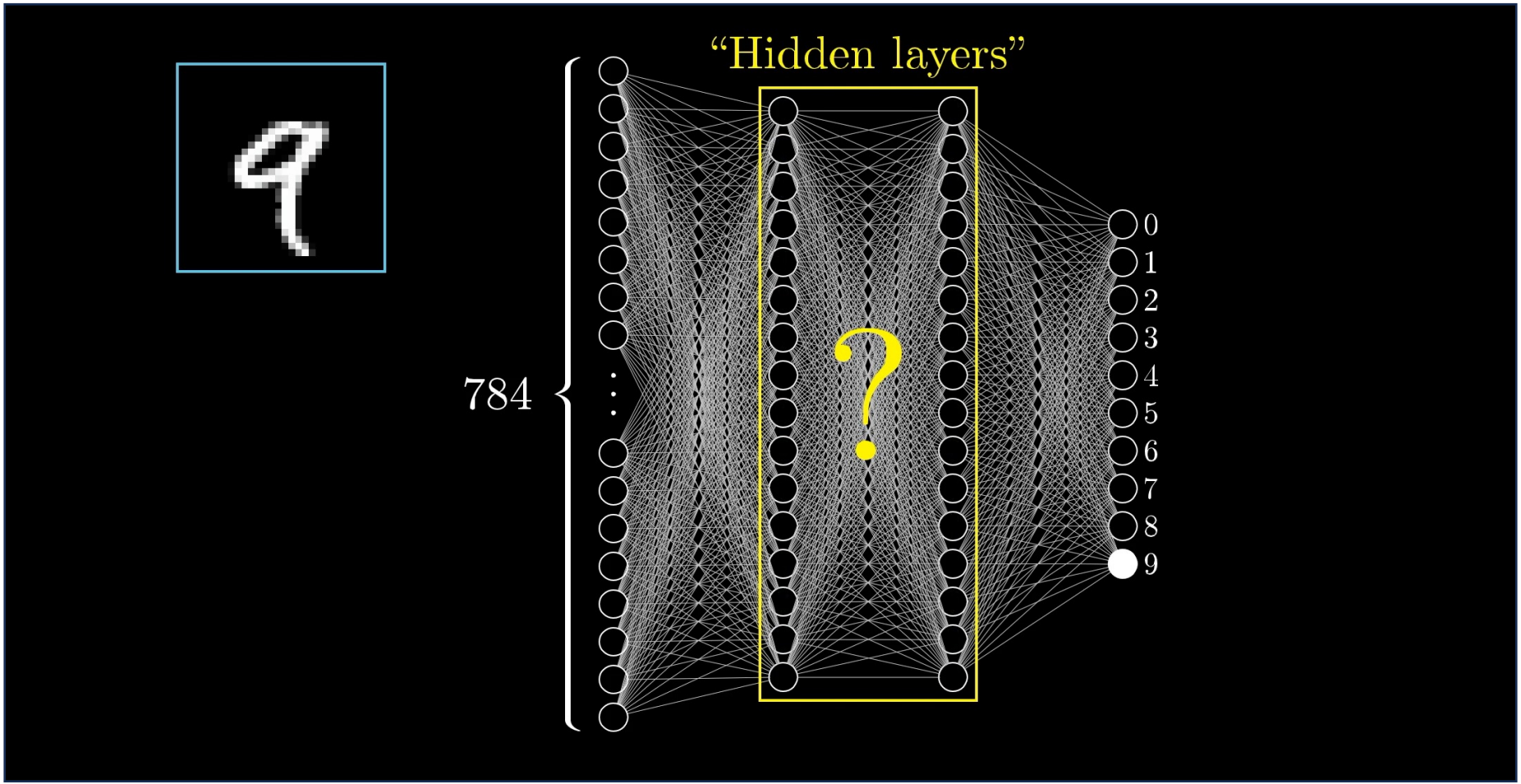


https://www.3blue1brown.com/lessons/neural-networks#title

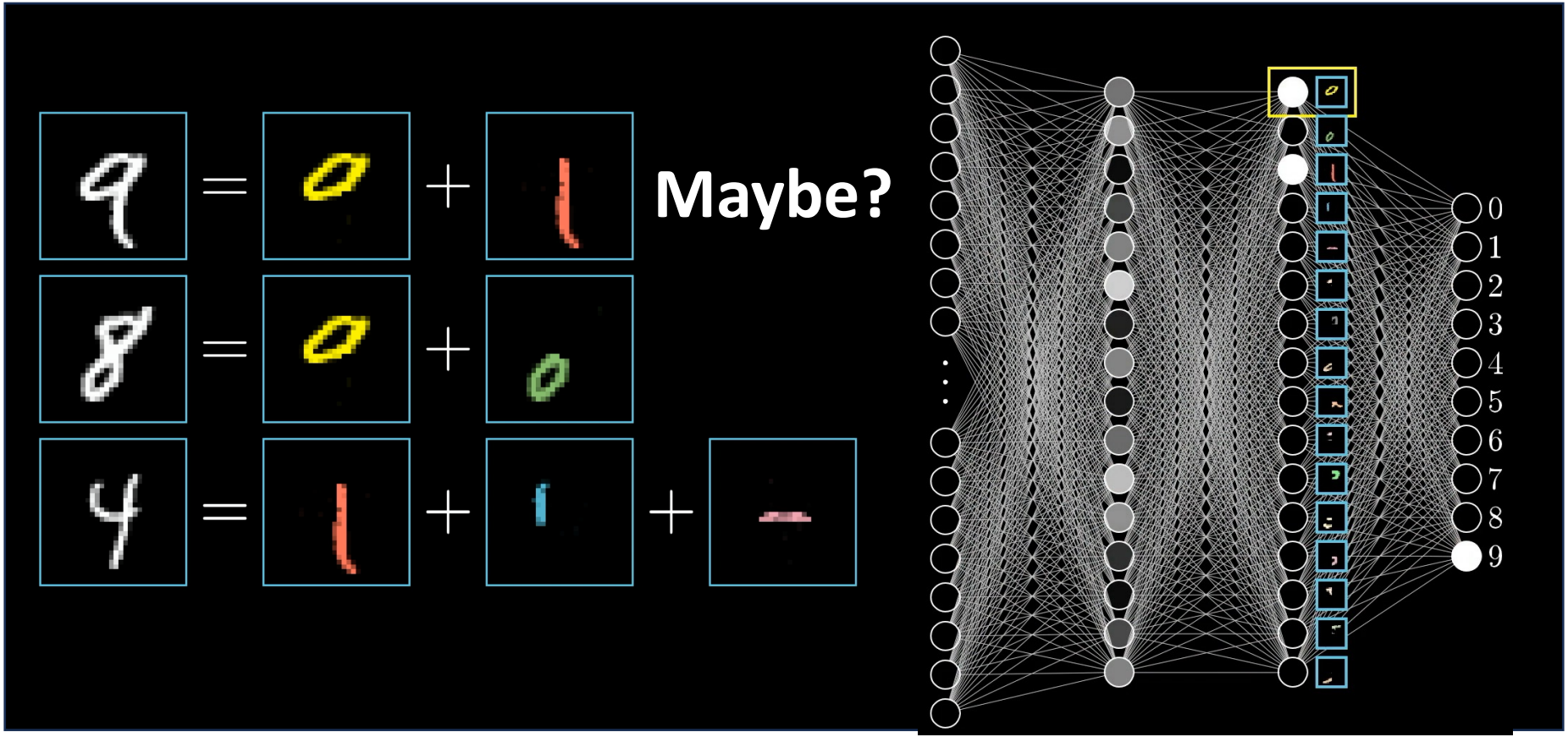
How does Neural Network learn?



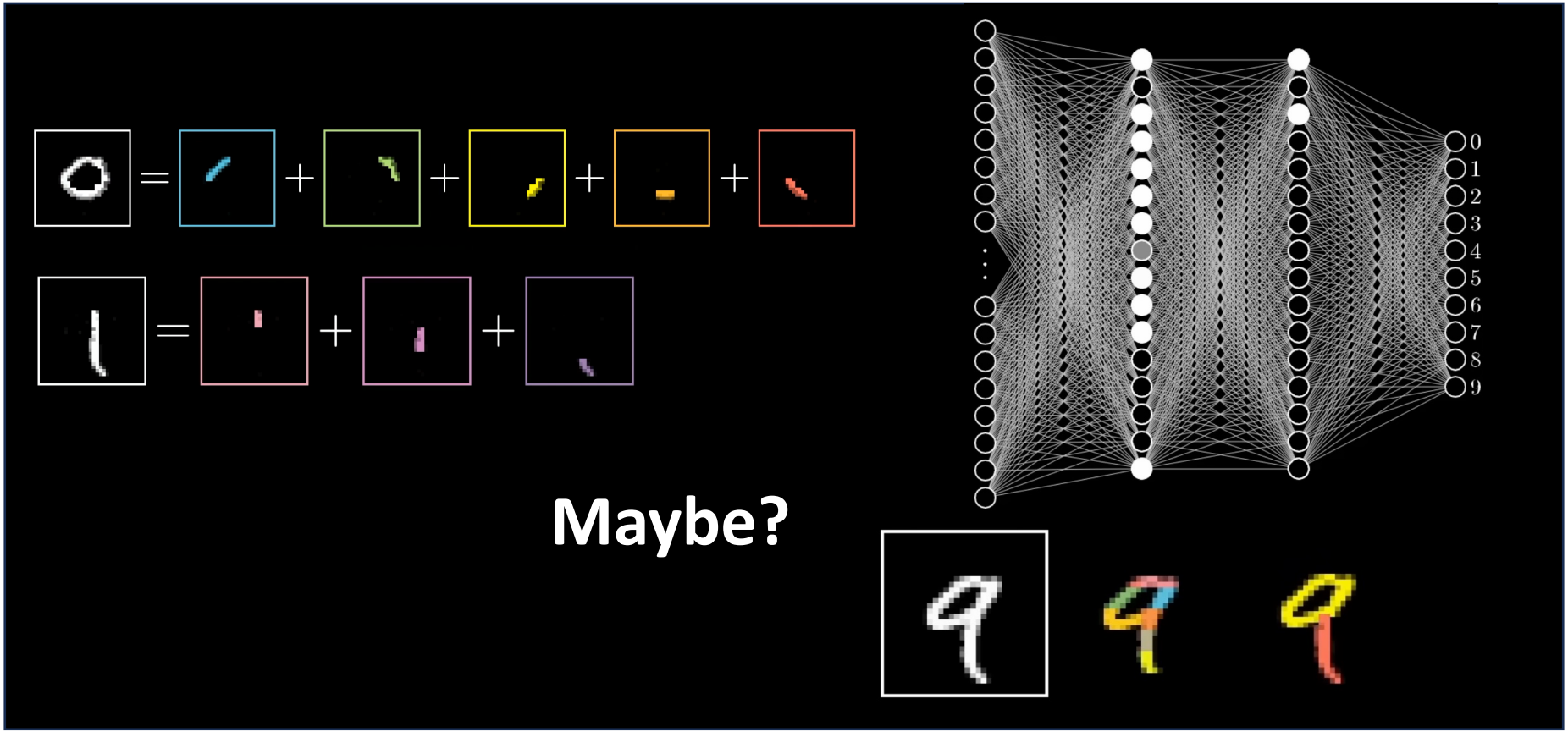

784
0
1
2
3
4
5
6
7
8
9


"Hidden layers"
784
?
0
1
2
3
4
5
6
7
8
9

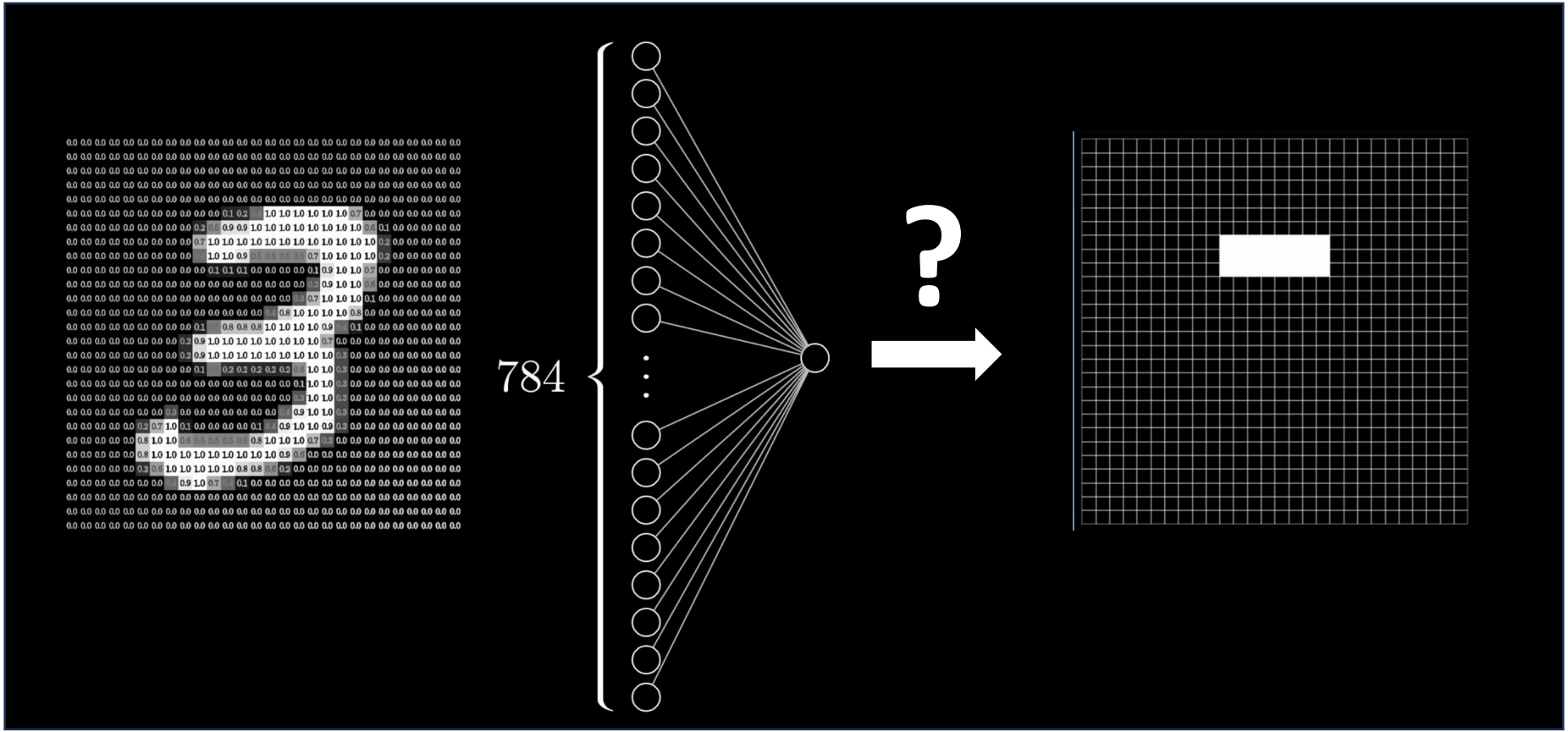
How does Neural Network learn?



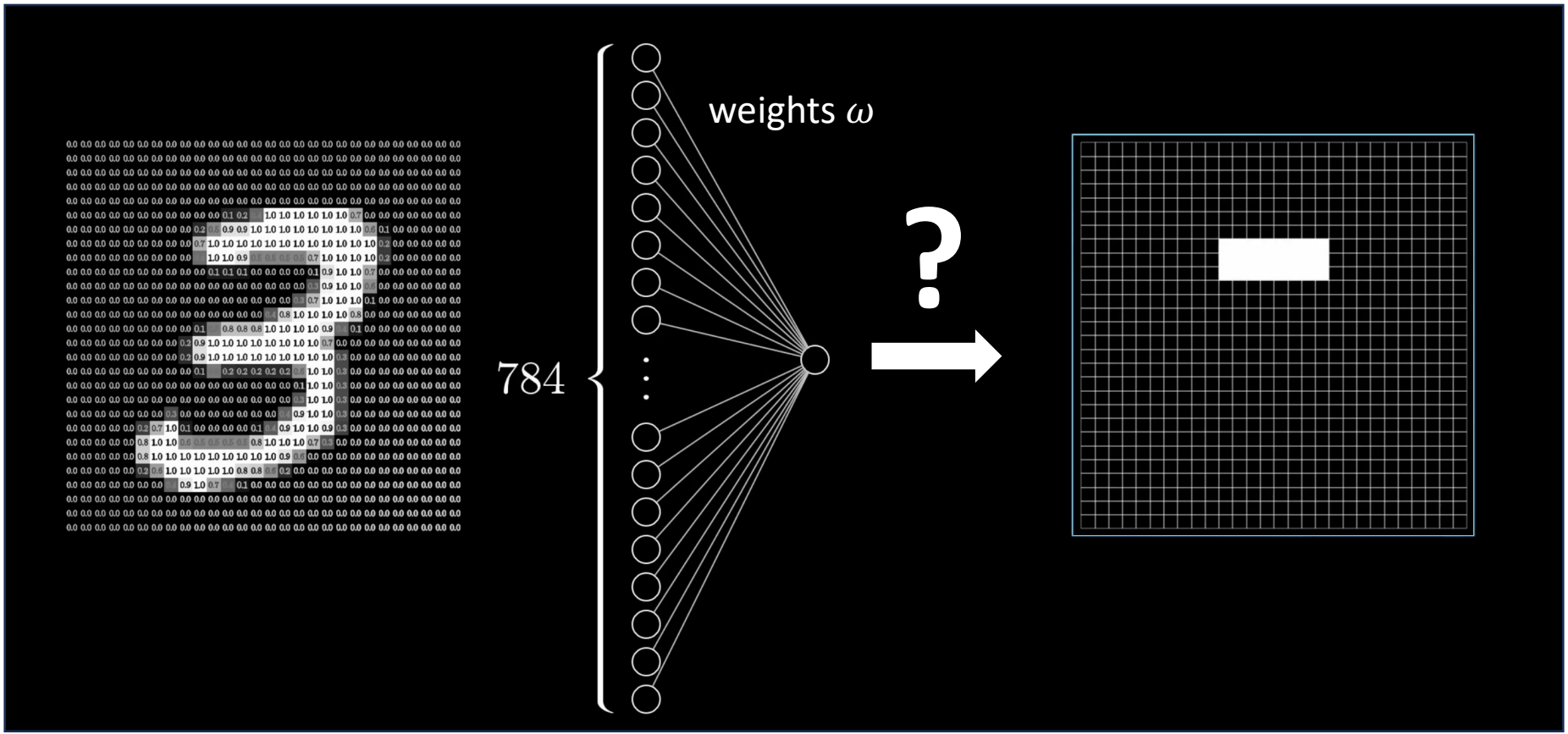
How does Neural Network learn?



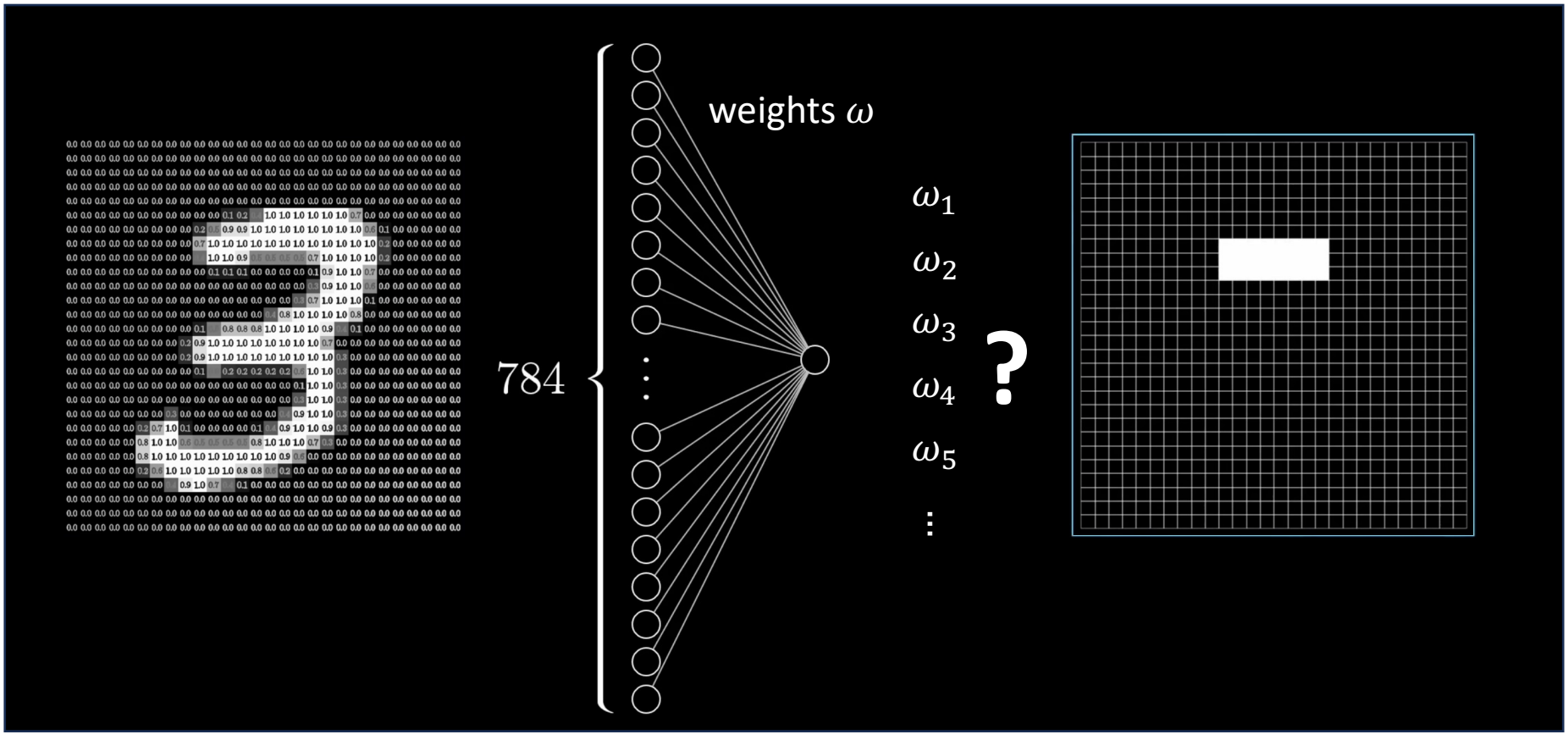
How Information Passes Between Layers



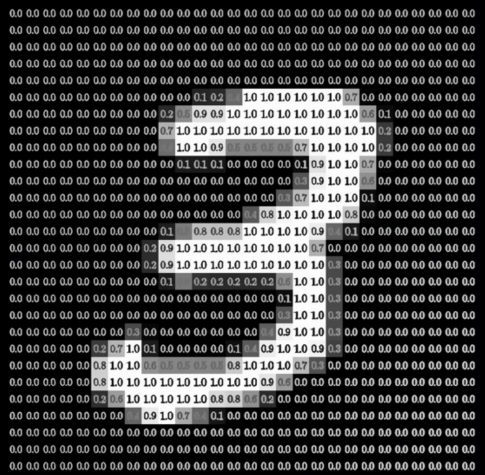
How Information Passes Between Layers



How Information Passes Between Layers



How Information Passes Between Layers



784

weights ω

input to neuron

$$z = \omega_1 a_1 + \omega_2 a_2 + \cdots + \omega_{784} a_{784}$$

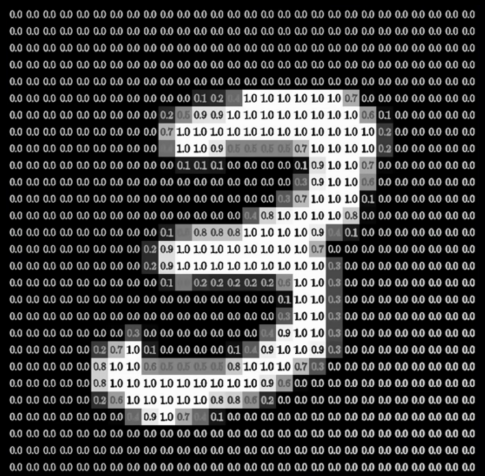
one output neuron N

Output of neuron:

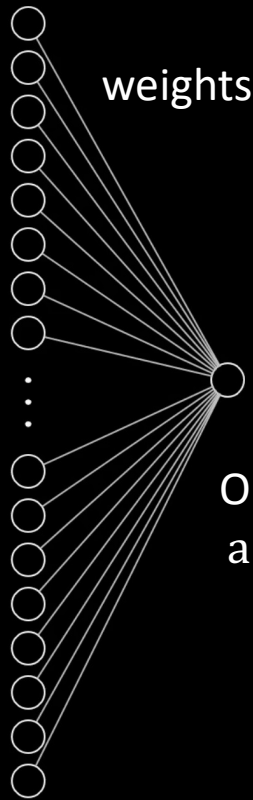
$$a = \sigma(\omega_1 a_1 + \omega_2 a_2 + \cdots + \omega_{784} a_{784})$$

sigmoid function $\sigma(x) = \frac{1}{1+e^{-x}}$

How Information Passes Between Layers



784



weights ω

input to neuron

$$z = \omega_1 a_1 + \omega_2 a_2 + \cdots + \omega_{784} a_{784}$$

one output neuron N

Output of neuron:

$$a = \sigma(\omega_1 a_1 + \omega_2 a_2 + \cdots + \omega_{784} a_{784} - 10)$$

"Bias"

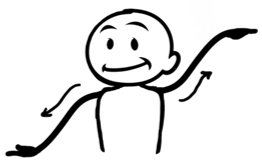
e.g., Only activate meaningfully when weighted sum > 10

Sigmoid



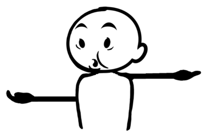
$$y = \frac{1}{1+e^{-x}}$$

Tanh



$$y = \tanh(x)$$

Step Function



$$y = \begin{cases} 0, & x<n \\ 1, & x \geq n \end{cases}$$

Softplus



$$y = \ln(1+e^{x})$$

ReLU



$$y = \begin{cases} 0, & x<0 \\ x, & x \geq 0 \end{cases}$$

Softsign



$$y = \frac{x}{(1+|x|)}$$

ELU



$$y = \begin{cases} \alpha(e^{x}-1), & x<0 \\ x, & x \geq 0 \end{cases}$$

Log of Sigmoid



$$y = \ln\left(\frac{1}{1+e^{-x}}\right)$$

Swish



$$y = \frac{x}{1+e^{-x}}$$

Sinc



$$y = \frac{\sin(x)}{x}$$

Leaky ReLU



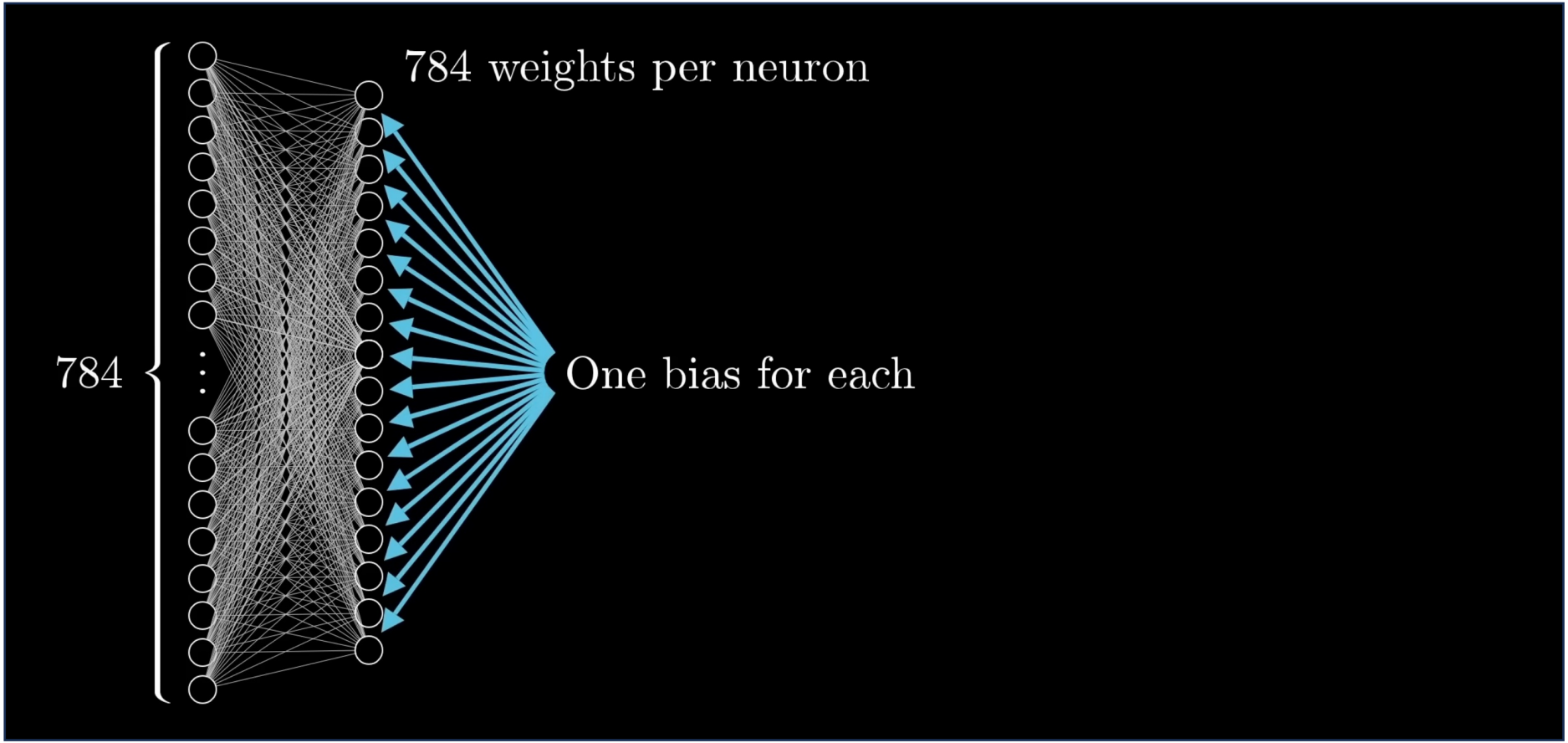
$$y = \max(0.1x, x)$$

Mish

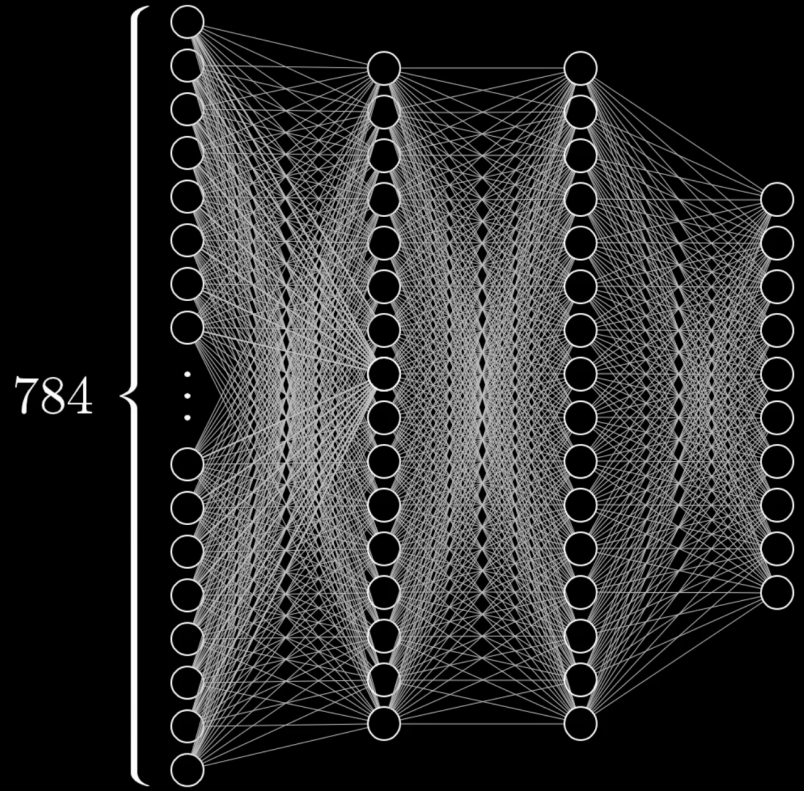


$$y = x(\tanh(\text{softplus}(x)))$$

How Information Passes Between Layers



How Information Passes Between Layers



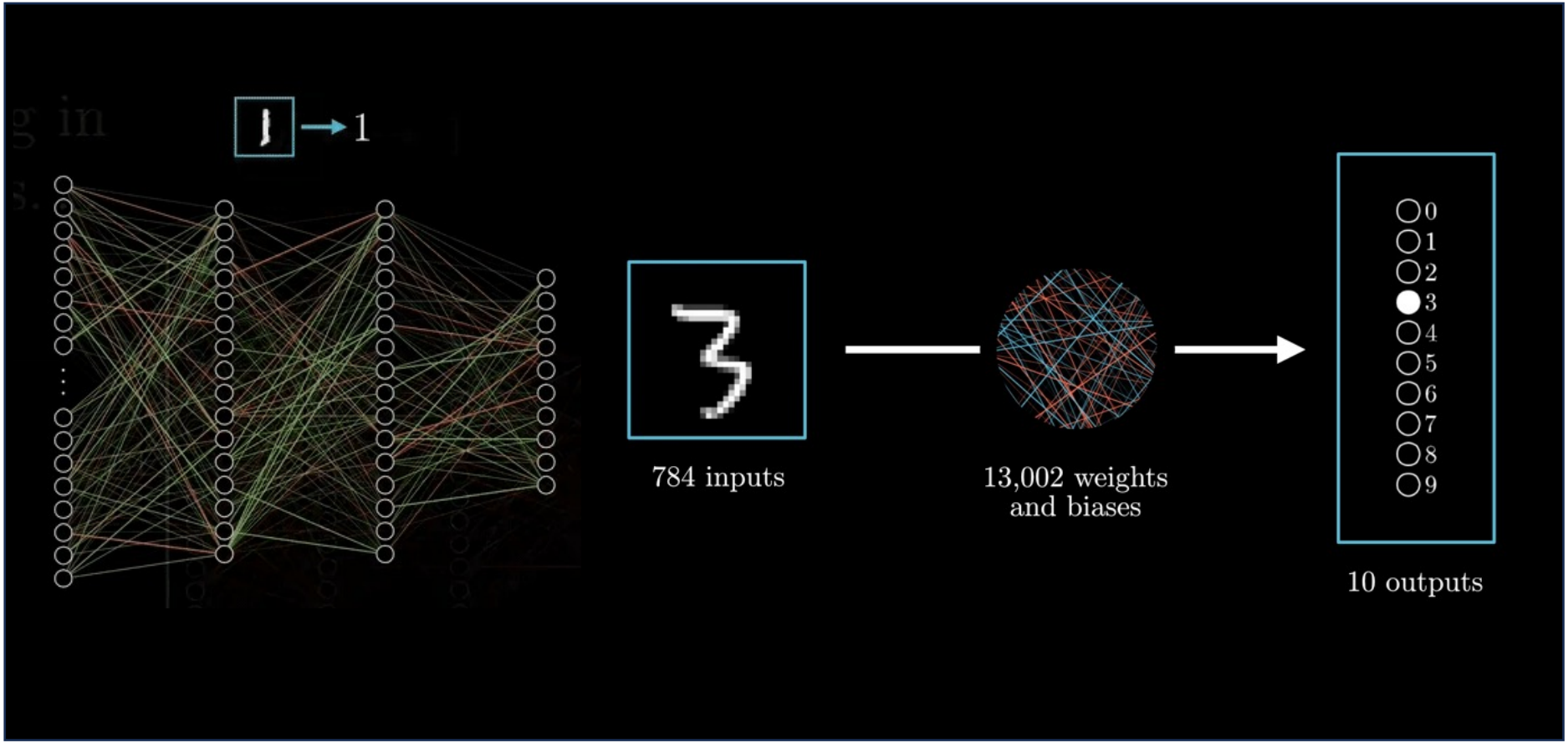
$$784 \times 16 + 16 \times 16 + 16 \times 10$$
weights

$$16 + 16 + 10$$
biases

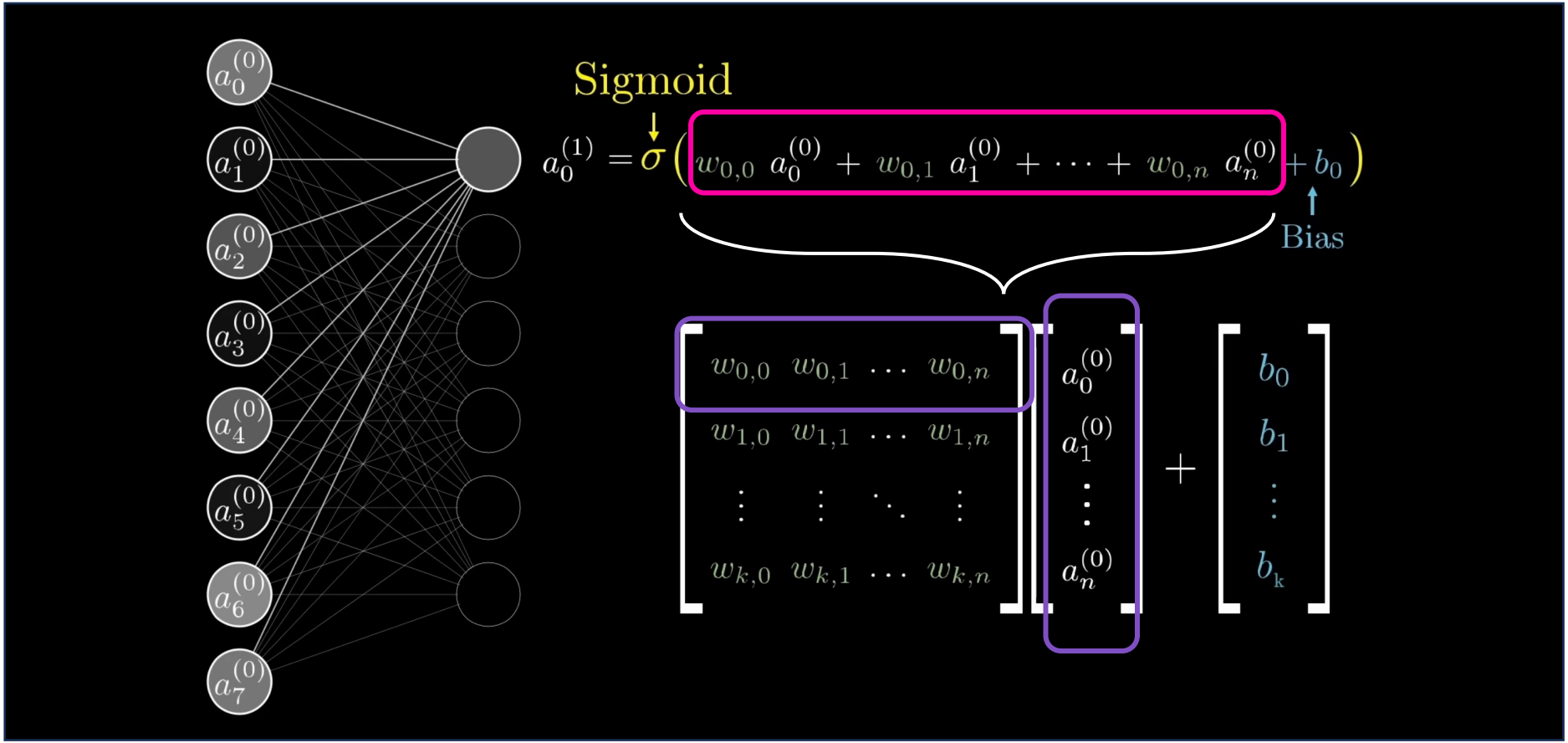
$$13{,}002$$

Finding the right weights and biases

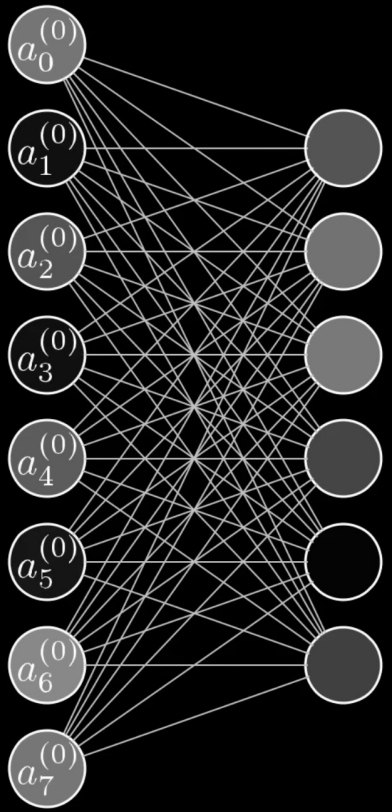
Network as a Function



How Information Passes Between Layers



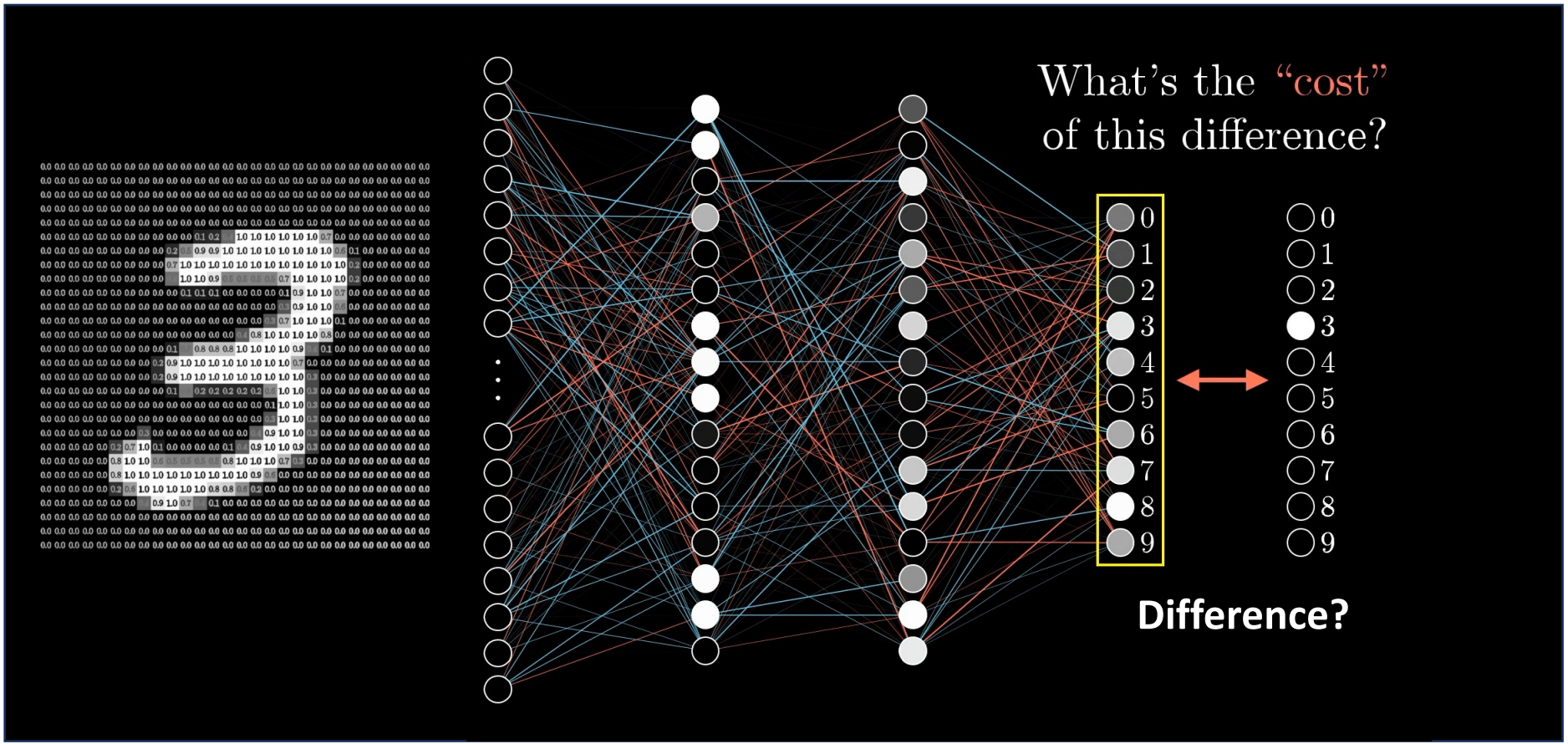
How Information Passes Between Layers



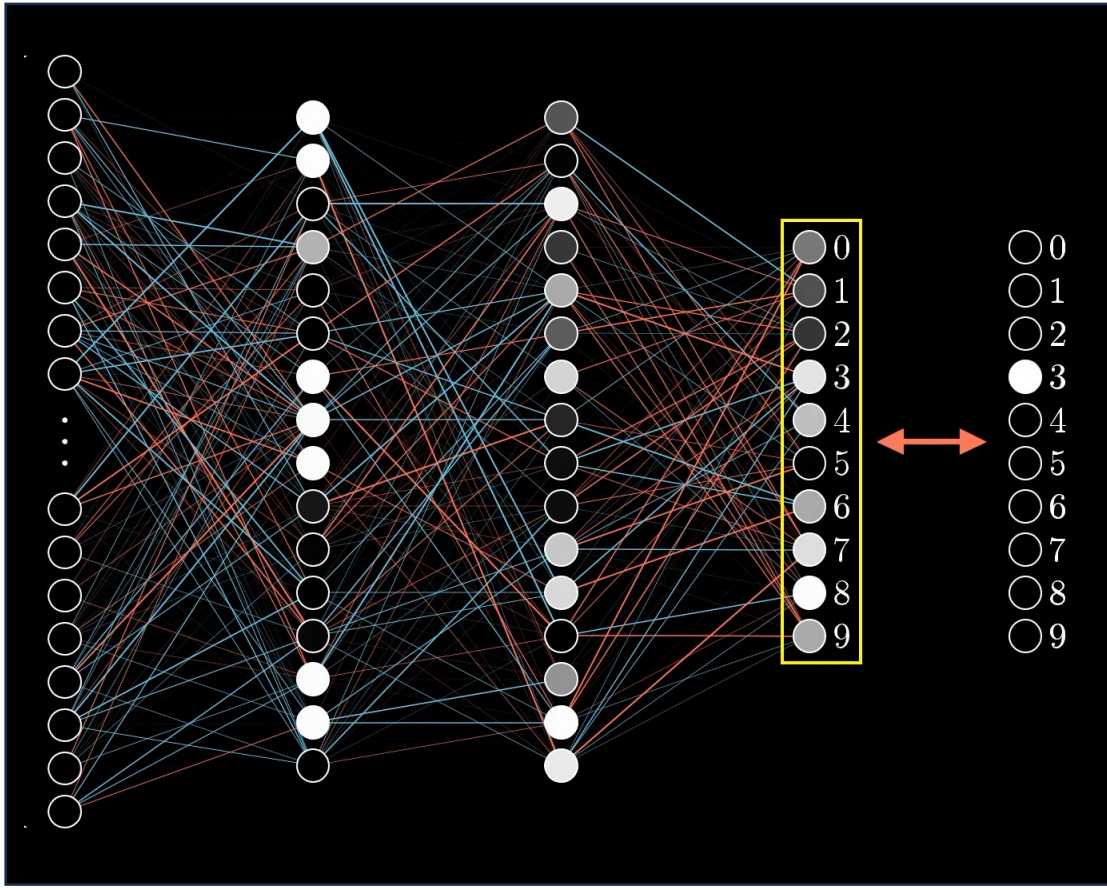
$$\mathbf{a}^{(1)} = \sigma(\mathbf{W}\mathbf{a}^{(0)} + \mathbf{b})$$

$$\sigma\left(\begin{bmatrix} w_{0,0} & w_{0,1} & \cdots & w_{0,n} \\ w_{1,0} & w_{1,1} & \cdots & w_{1,n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{k,0} & w_{k,1} & \cdots & w_{k,n} \end{bmatrix} \begin{bmatrix} a_0^{(0)} \\ a_1^{(0)} \\ \vdots \\ a_n^{(0)} \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \\ \vdots \\ b_k \end{bmatrix}\right)$$

How does Neural Network learn?



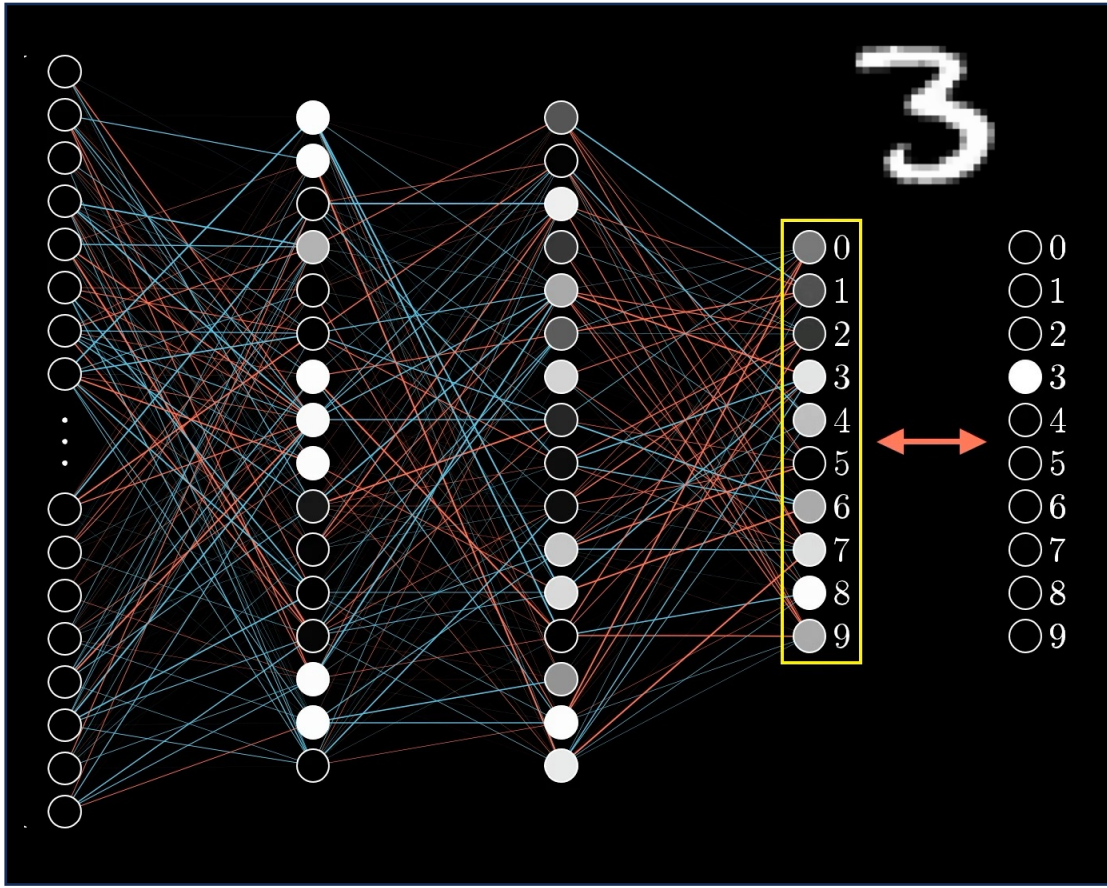
How does Neural Network learn?



Difference?

$a_j^{(L)}$	y_j
0.22	0.00
0.57	0.00
0.01	0.00
0.92	**1.00**
0.46	0.00
0.93	0.00
0.11	0.00
1.00	0.00
0.21	0.00
0.32	0.00

How does Neural Network learn?

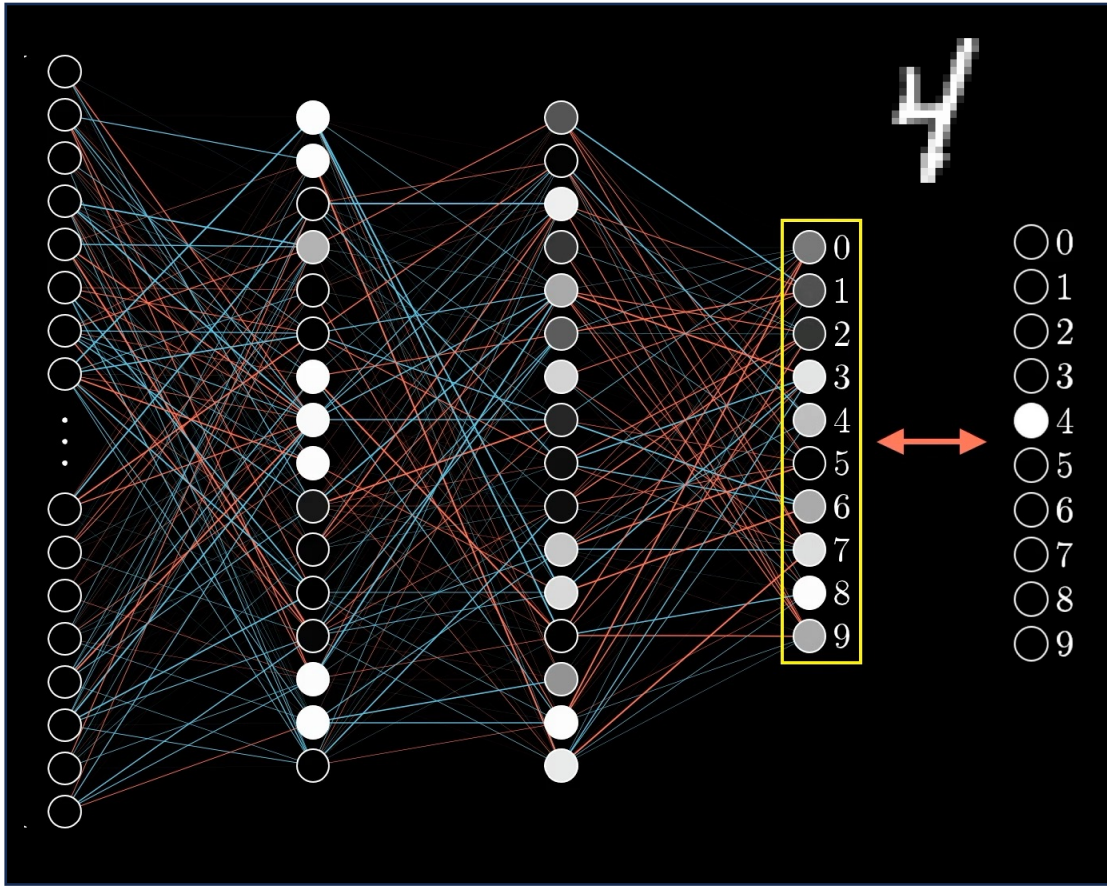


Mean square error

$$(0.22 - 0.00)^2$$
$$(0.57 - 0.00)^2$$
$$(0.01 - 0.00)^2$$
$$(\mathbf{0.92 - 1.00})^2$$
$$(0.46 - 0.00)^2$$
$$(0.93 - 0.00)^2$$
$$(0.11 - 0.00)^2$$
$$(1.00 - 0.00)^2$$
$$(0.21 - 0.00)^2$$
$$(0.32 - 0.00)^2$$

$$C_0 = \sum_{j=0}^{9} \left(a_j^{(L)} - y_j\right)^2$$

How does Neural Network learn?



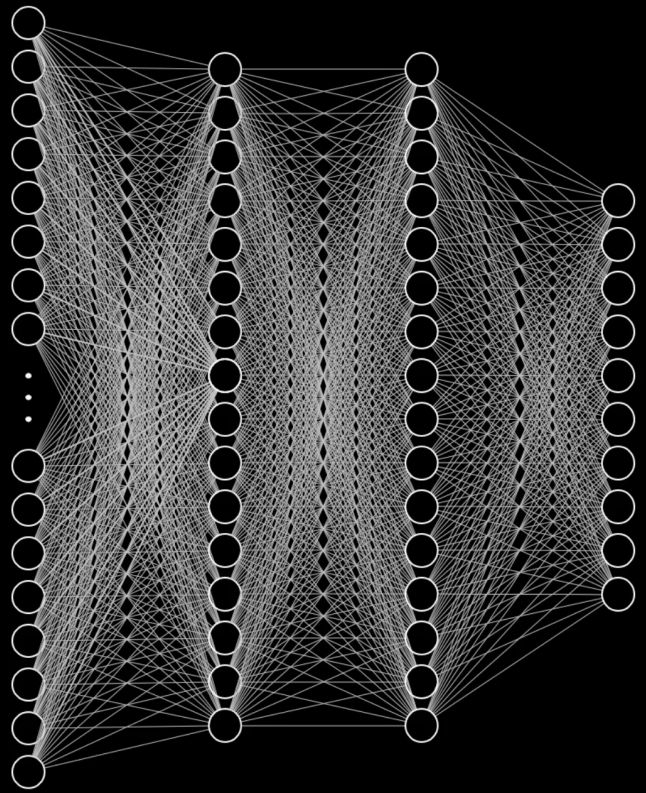
Mean square error

$$(0.22 - 0.00)^2$$
$$(0.57 - 0.00)^2$$
$$(0.01 - 0.00)^2$$
$$(0.62 - 0.00)^2$$
$$(\mathbf{0.46 - 1.00})^2$$
$$(0.93 - 0.00)^2$$
$$(0.11 - 0.00)^2$$
$$(1.00 - 0.00)^2$$
$$(0.21 - 0.00)^2$$
$$(0.32 - 0.00)^2$$

$$C_1 = \sum_{j=0}^{9} \left(a_j^{(L)} - y_j\right)^2$$

How does Neural Network learn?

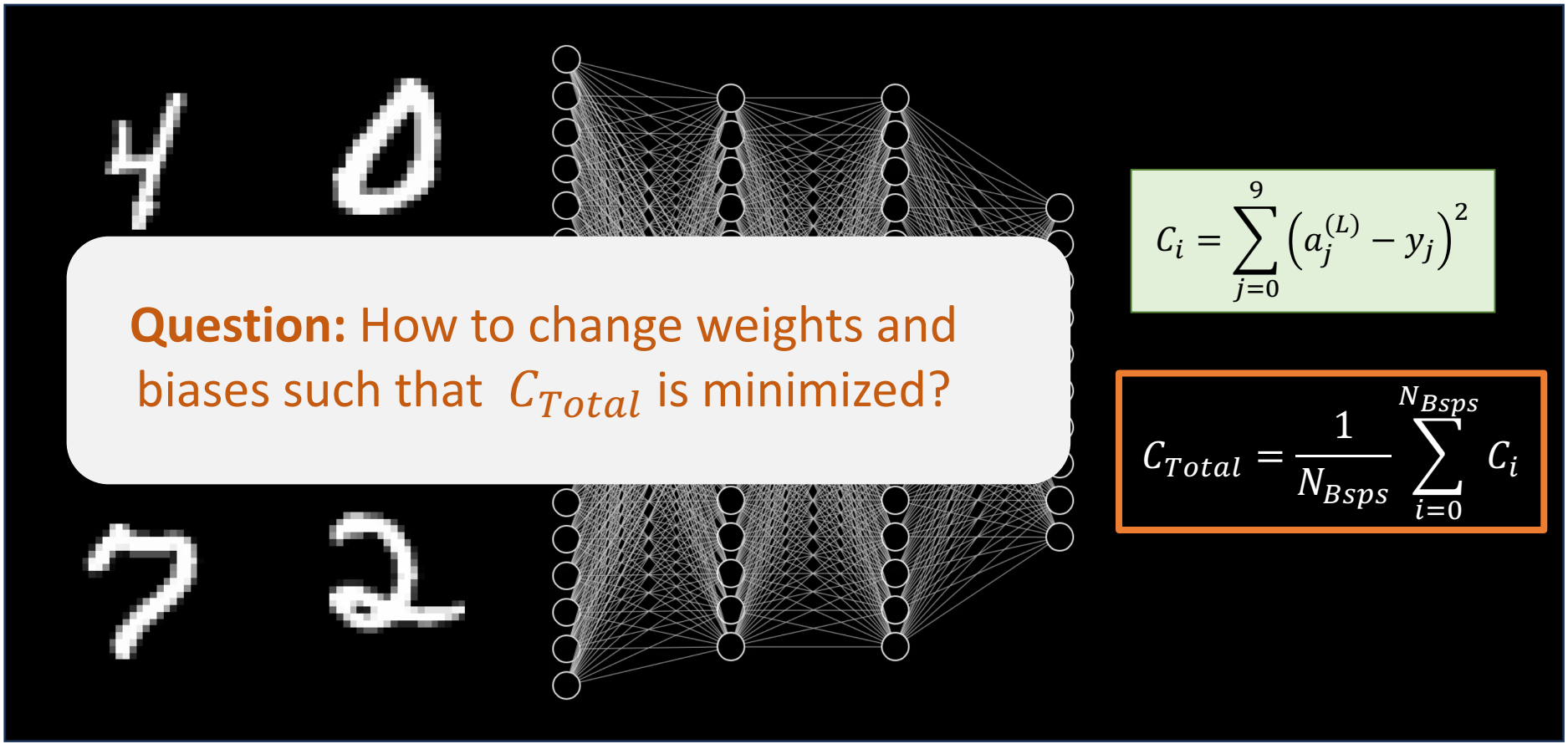




$$C_i = \sum_{j=0}^{9} \left(a_j^{(L)} - y_j\right)^2$$

$$C_{Total} = \frac{1}{N_{Bsps}} \sum_{i=0}^{N_{Bsps}} C_i$$

How does Neural Network learn?



Question: How to change weights and biases such that C_{Total} is minimized?

$$C_i = \sum_{j=0}^{9} \left(a_j^{(L)} - y_j\right)^2$$

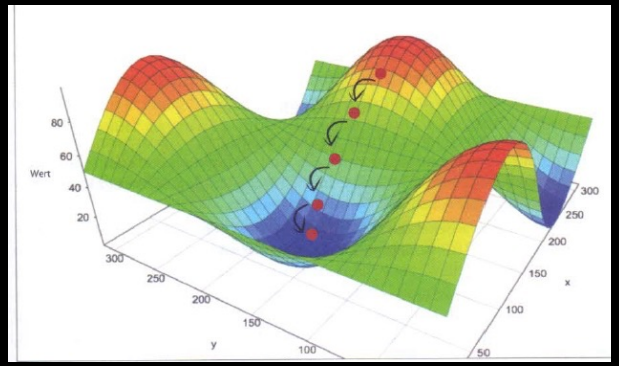
$$C_{Total} = \frac{1}{N_{Bsps}} \sum_{i=0}^{N_{Bsps}} C_i$$

How does Neural Network learn?

$$C_i = \sum_{j=0}^{9} \left(a_j^{(L)} - y_j\right)^2$$

$$C_{Total} = \frac{1}{N_{Bsps}} \sum_{i=0}^{N_{Bsps}} C_i$$

Gradient descent



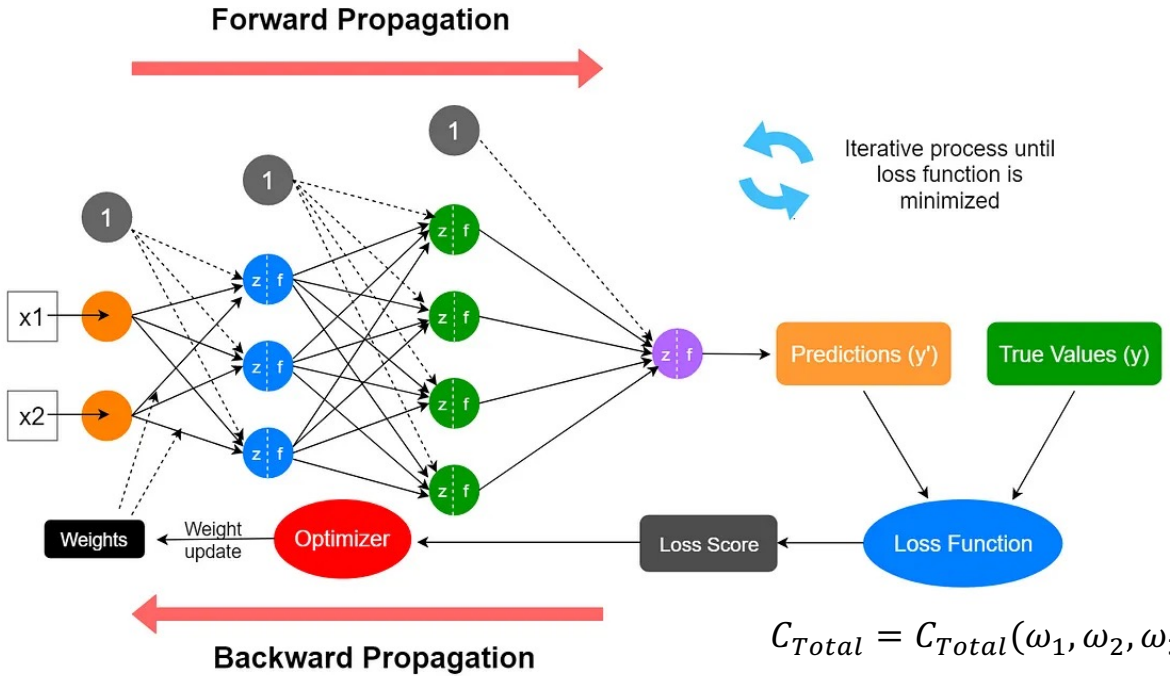
Question: How to change weights and biases such that C_{Total} is minimized?

$$C_{Total} = C_{Total}(\omega_1, \omega_2, \omega_3, \ldots, b_1, b_2, b_3)$$

Question: How to compute $-\nabla C_{Total}$?

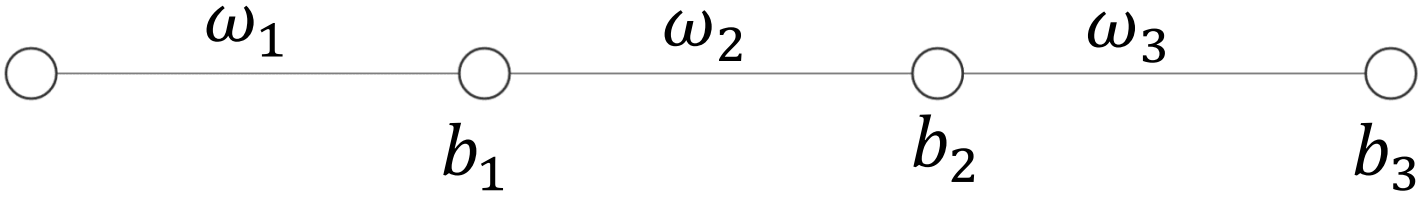
How does Neural Network learn?

Backpropagation is the algorithm that neural networks use to adjust their weights by computing how much each weight contributed to the error (a gradient computation method commonly used for training a neural network in computing parameter updates).



$$C_{Total} = C_{Total}(\omega_1, \omega_2, \omega_3, \ldots, b_1, b_2, b_3)$$

cost function: $C_0(\omega_1, \omega_2, \omega_3, b_1, b_2, b_3)$



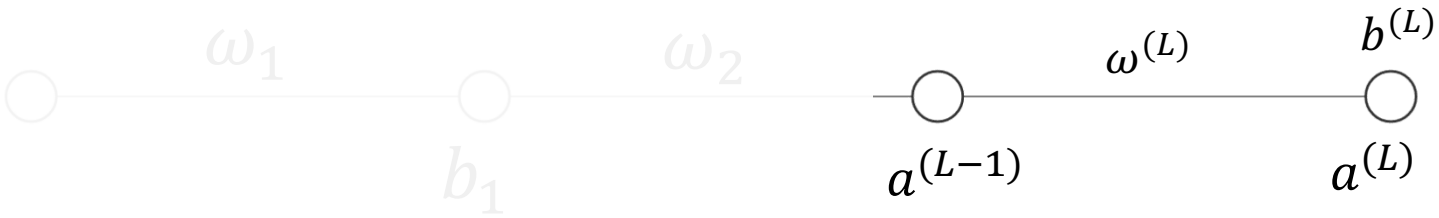
cost function: $C_0(\omega_1, \omega_2, \omega_3, b_1, b_2, b_3)$



desired result: y

cost function: $C_0(\dots) = \left(a^{(L)} - y\right)^2$

$$a^{(L)} = \sigma\left(\omega^{(L)} a^{(L-1)} + b^{(L)}\right)$$

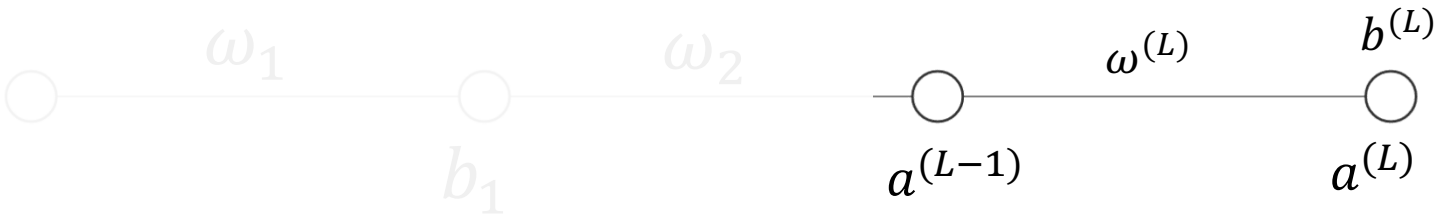


desired result: y

cost function: $C_0(\dots) = \left(a^{(L)} - y\right)^2$

input into neuron L: $z^{(L)} = \omega^{(L)} a^{(L-1)} + b^{(L)}$

output of neuron L: $a^{(L)} = \sigma\left(z^{(L)}\right)$

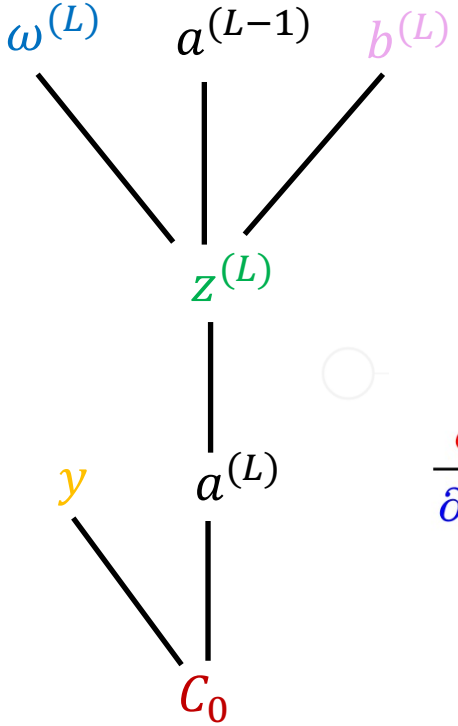


desired result: y

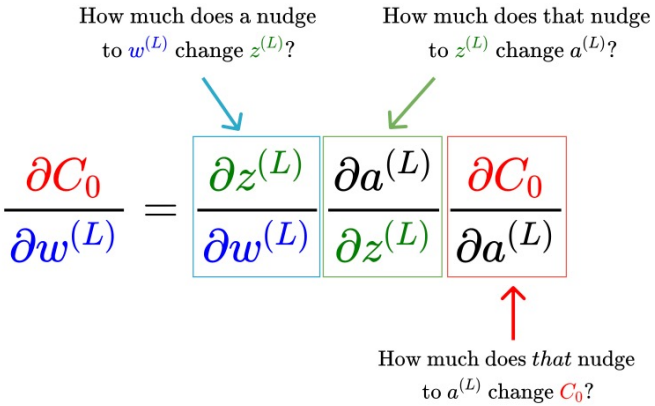
cost function: $C_0(\dots) = \left(a^{(L)} - y\right)^2$

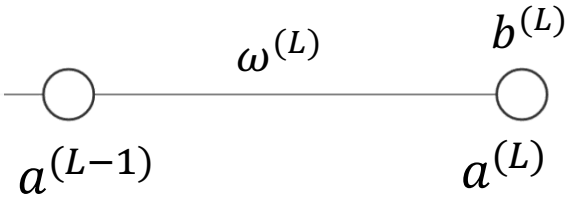
input into neuron L: $z^{(L)} = \omega^{(L)} a^{(L-1)} + b^{(L)}$

output of neuron L: $a^{(L)} = \sigma\left(z^{(L)}\right)$



chain rule





desired result: y

$$\frac{\partial C_0}{\partial w^{(L)}} = \frac{\partial z^{(L)}}{\partial w^{(L)}} \frac{\partial a^{(L)}}{\partial z^{(L)}} \frac{\partial C_0}{\partial a^{(L)}}$$

$$z^{(L)} = w^{(L)} a^{(L-1)} + b^{(L)} \longrightarrow \frac{\partial z^{(L)}}{\partial w^{(L)}} = a^{(L-1)}$$

$$a^{(L)} = \sigma\left(z^{(L)}\right) \longrightarrow \frac{\partial a^{(L)}}{\partial z^{(L)}} = \sigma'\left(z^{(L)}\right)$$

$$C_0 = \left(a^{(L)} - y\right)^2 \longrightarrow \frac{\partial C_0}{\partial a^{(L)}} = 2\left(a^{(L)} - y\right)$$

Weights

$$\frac{\partial C_0}{\partial w^{(L)}} = a^{(L-1)} \sigma'\left(z^{(L)}\right) 2\left(a^{(L)} - y\right)$$

The Cost for All Training Data

$$C = \frac{1}{n} \sum_{k=0}^{n-1} C_k$$

$$\frac{\partial C}{\partial w^{(L)}} = \frac{1}{n} \sum_{k=0}^{n-1} \frac{\partial C_k}{\partial w^{(L)}}$$

$$\frac{\partial C_0}{\partial b^{(L)}} = \frac{\partial z^{(L)}}{\partial b^{(L)}} \frac{\partial a^{(L)}}{\partial z^{(L)}} \frac{\partial C_0}{\partial a^{(L)}}$$

$$z^{(L)} = w^{(L)} a^{(L-1)} + b^{(L)} \longrightarrow \frac{\partial z^{(L)}}{\partial b^{(L)}} = 1$$

$$a^{(L)} = \sigma\left(z^{(L)}\right) \longrightarrow \frac{\partial a^{(L)}}{\partial z^{(L)}} = \sigma'\left(z^{(L)}\right)$$

$$C_0 = \left(a^{(L)} - y\right)^2 \longrightarrow \frac{\partial C_0}{\partial a^{(L)}} = 2\left(a^{(L)} - y\right)$$

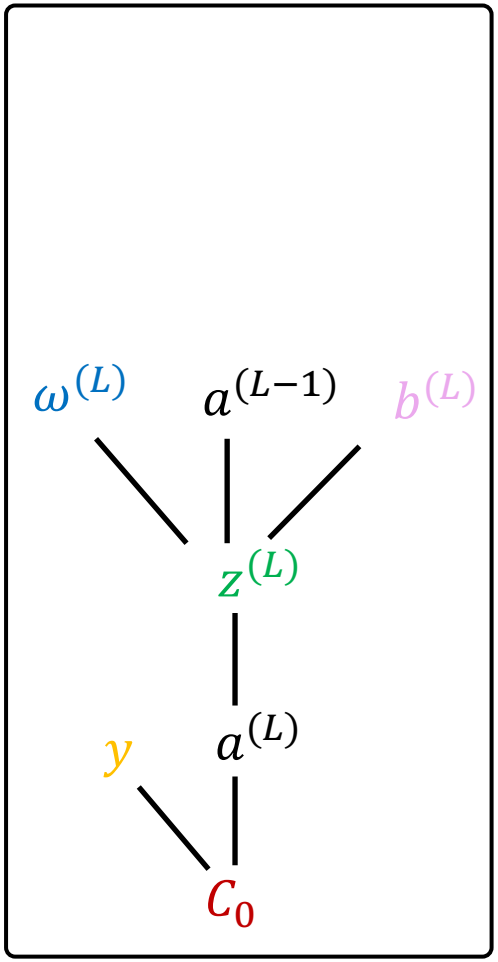
Biases

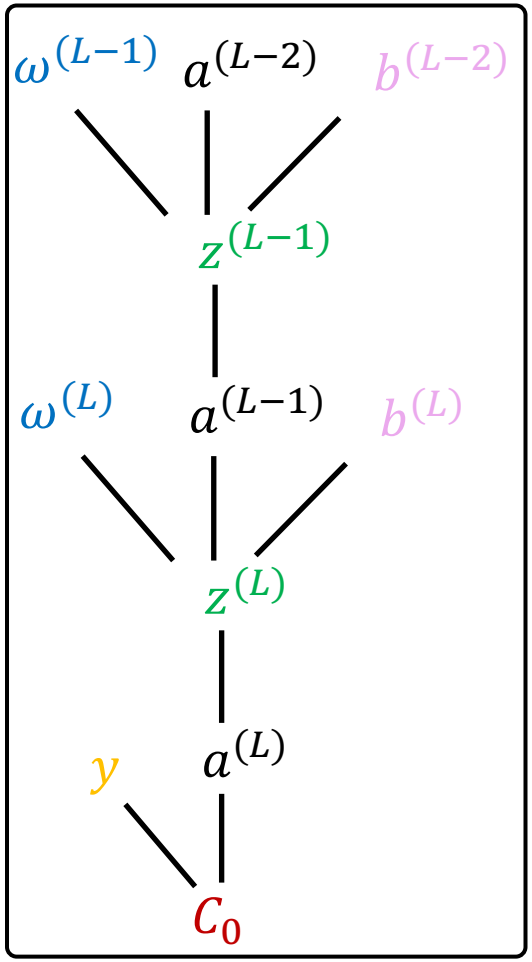
$$\frac{\partial C_0}{\partial w^{(L)}} = a^{(L-1)} \sigma'\left(z^{(L)}\right) 2\left(a^{(L)} - y\right)$$

The Cost for All Training Data

$$C = \frac{1}{n} \sum_{k=0}^{n-1} C_k$$

$$\frac{\partial C}{\partial b^{(L)}} = \frac{1}{n} \sum_{k=0}^{n-1} \frac{\partial C_k}{\partial b^{(L)}}$$





cost function: $C_0(\dots) = \left(a^{(L)} - y\right)^2$

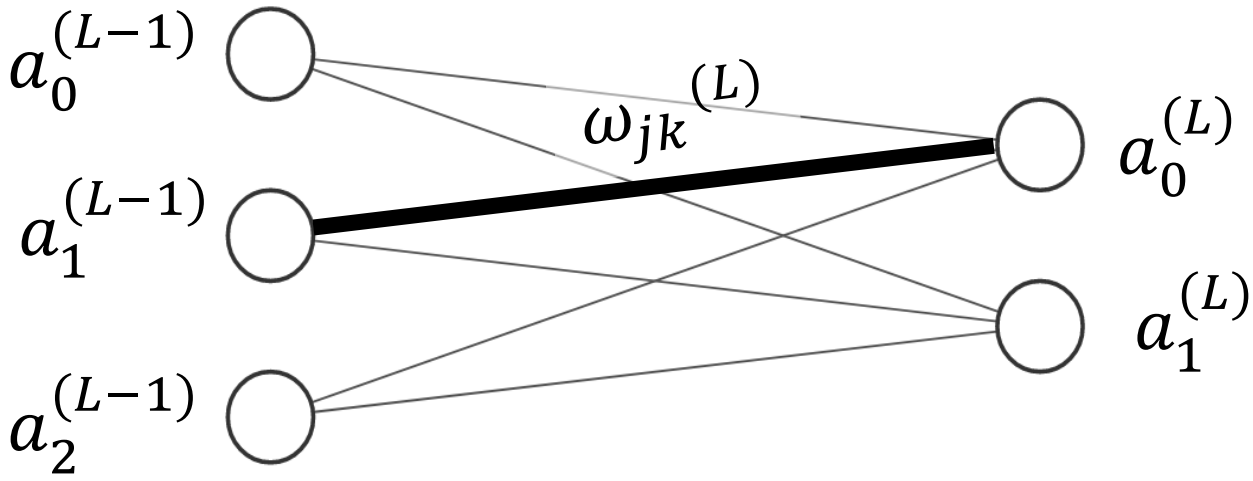
$$z^{(L)} = \omega^{(L)} a^{(L-1)} + b^{(L)}$$

$$a^{(L)} = \sigma\left(z^{(L)}\right)$$

$$\frac{\partial C_0}{\partial a^{(L-1)}} = \frac{\partial z^{(L)}}{\partial a^{(L-1)}} \frac{\partial a^{(L)}}{\partial z^{(L)}} \frac{\partial C_0}{\partial a^{(L)}}$$

$$\frac{\partial C_0}{\partial \omega^{(L-1)}} = \frac{\partial z^{(L-1)}}{\partial \omega^{(L-1)}} \frac{\partial a^{(L-1)}}{\partial z^{(L-1)}} \frac{\partial C_0}{\partial a^{(L-1)}}$$

cost function: $C_0 = \sum_{j=0}^{n_L-1} \left(a_j{}^{(L)} - y_j\right)^2$



desired result

y_0

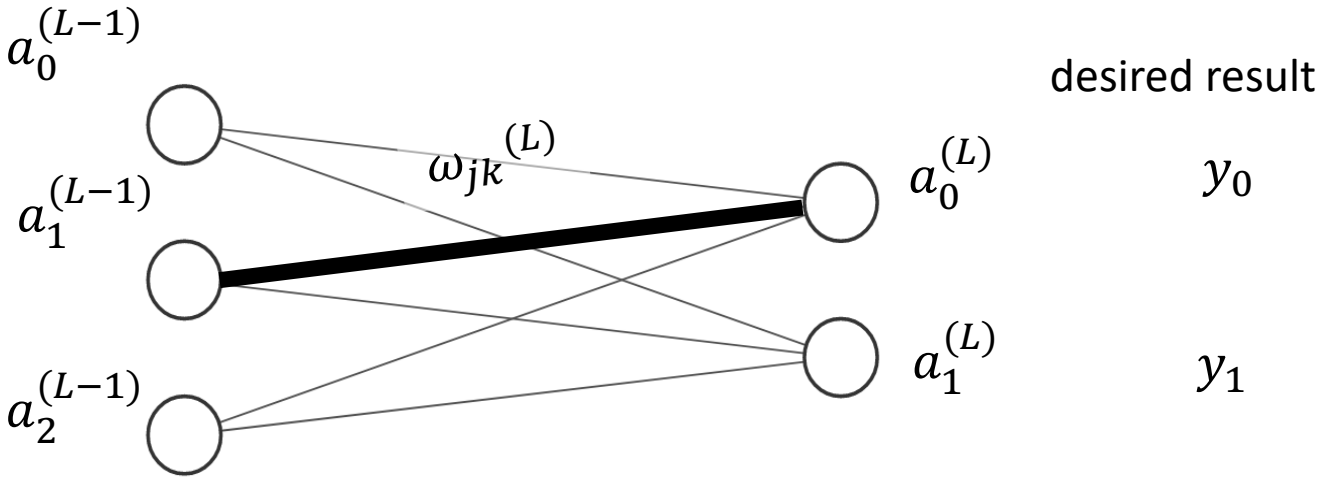
y_1

cost function: $C_0 = \sum_{j=0}^{n_L-1} \left(a_j{}^{(L)} - y_j\right)^2$

$$\frac{\partial C_0}{\partial \omega_{jk}{}^{(L)}} = \frac{\partial z_j{}^{(L)}}{\partial \omega_{jk}{}^{(L)}} \frac{\partial a_j{}^{(L)}}{\partial z_j{}^{(L)}} \frac{\partial C_0}{\partial a_j{}^{(L)}}$$

$z_j{}^{(L)} = \omega_{j0}{}^{(L)} a_0{}^{(L-1)} + \omega_{j1}{}^{(L)} a_1{}^{(L-1)} + \omega_{j2}{}^{(L)} a_2{}^{(L-1)} + b_j$

$a_j{}^{(L)} = \sigma(z_j{}^{(L)})$



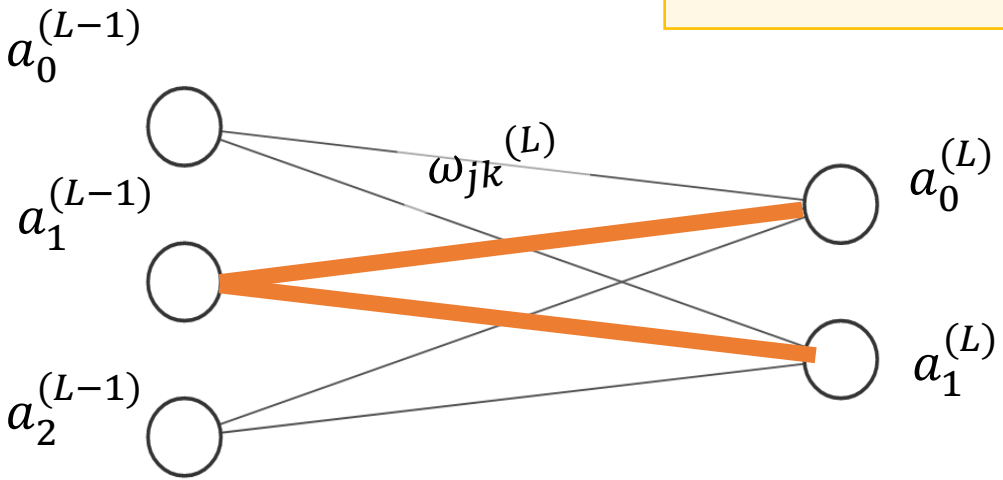
cost function: $C_0 = \sum_{j=0}^{n_L-1} \left(a_j^{(L)} - y_j\right)^2$

$$z_j^{(L)} = \omega_{j0}^{(L)} a_0^{(L-1)} + \omega_{j1}^{(L)} a_1^{(L-1)} + \omega_{j2}^{(L)} a_2^{(L-1)} + b_j$$

$$a_j^{(L)} = \sigma\left(z_j^{(L)}\right)$$

$$\frac{\partial C_0}{\partial \omega_{jk}^{(L)}} = \frac{\partial z_j^{(L)}}{\partial \omega_{jk}^{(L)}} \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} \frac{\partial C_0}{\partial a_j^{(L)}}$$

$$\frac{\partial C_0}{\partial a_k^{(L-1)}} = \sum_{j=0}^{n_L-1} \frac{\partial z_j^{(L)}}{\partial a_k^{(L-1)}} \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} \frac{\partial C_0}{\partial a_j^{(L)}}$$



desired result

y_0

y_1

cost function: $C_0 = \sum_{j=0}^{n_L-1} \left(a_j^{(L)} - y_j\right)^2$

$$z_j^{(L)} = \omega_{j0}^{(L)} a_0^{(L-1)} + \omega_{j1}^{(L)} a_1^{(L-1)} + \omega_{j2}^{(L)} a_2^{(L-1)} + b_j$$

$$a_j^{(L)} = \sigma(z_j^{(L)})$$

$$\frac{\partial C_0}{\partial \omega_{jk}^{(L)}} = \frac{\partial z_j^{(L)}}{\partial \omega_{jk}^{(L)}} \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} \frac{\partial C_0}{\partial a_j^{(L)}}$$

$$\frac{\partial C_0}{\partial a_k^{(L-1)}} = \sum_{j=0}^{n_L-1} \frac{\partial z_j^{(L)}}{\partial a_k^{(L-1)}} \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} \frac{\partial C_0}{\partial a_j^{(L)}}$$

Sum over layer L

$$\nabla C_0 \begin{cases} \frac{\partial C_0}{\partial \omega_{jk}^{(l)}} = \underbrace{\frac{\partial z_j^{(l)}}{\partial \omega_{jk}^{(l)}}}_{= a_k^{(l-1)}} \underbrace{\frac{\partial a_j^{(l)}}{\partial z_j^{(l)}}}_{\sigma'(z_j^{(l)})} \frac{\partial C_0}{\partial a_j^{(l)}} \\ \text{Same for biases} \end{cases}$$

$$2(a_j^{(L)} - y_j)$$

or

$$\sum_{j=0}^{n_L-1} \omega_{jk}^{(L)} \; \sigma'(z_j^{(L+1)}) \; \frac{\partial C_0}{\partial a_j^{(l+1)}}$$

$$\nabla C_0 \longrightarrow \nabla C = \frac{1}{N}\sum_{i=0}^{N-1} \nabla C_i$$

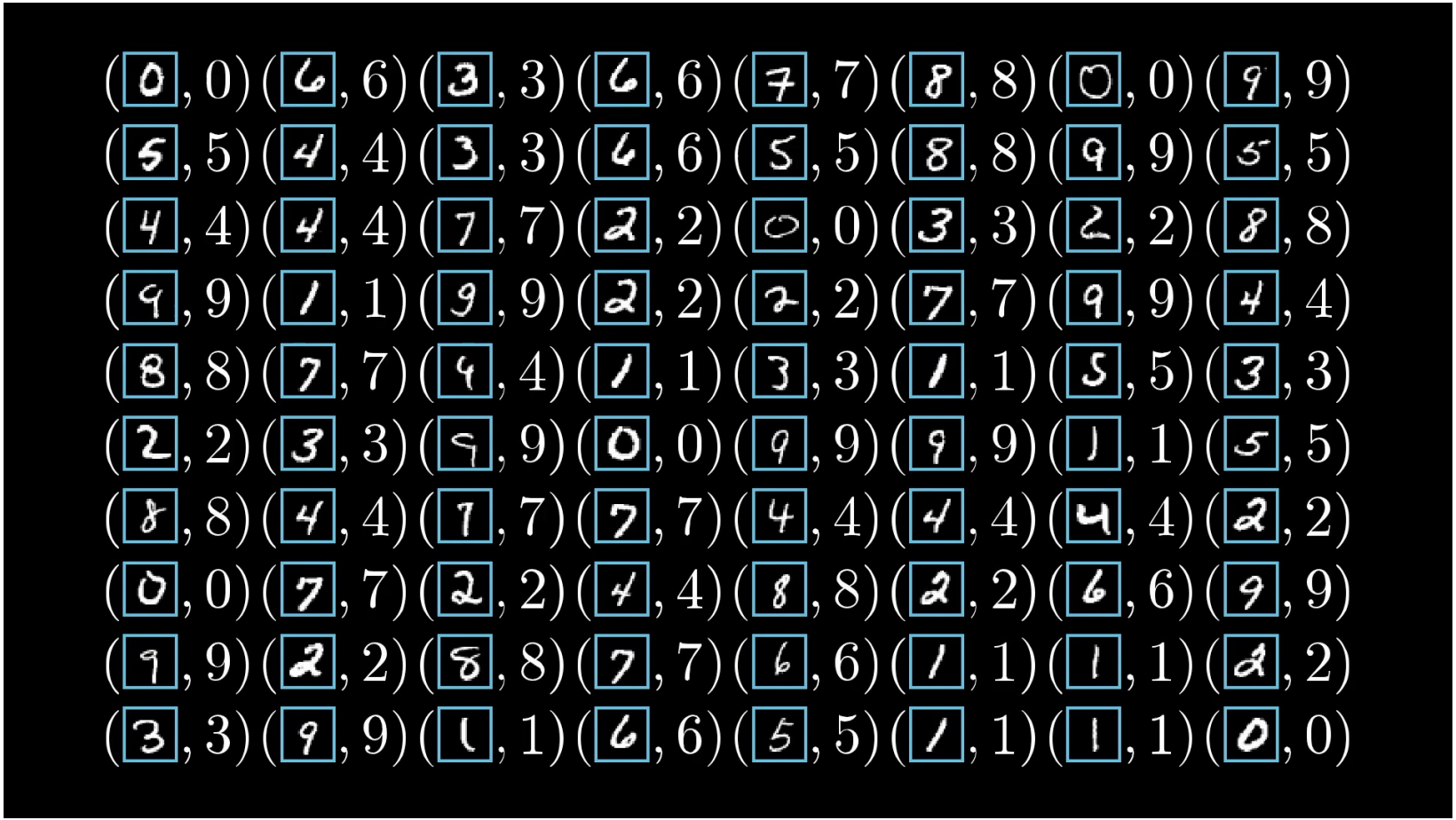
Expensive to compute for N examples

$$\nabla C \approx \frac{1}{m}\sum_{i=0}^{m-1} \nabla C_{rnd(i)}$$

Approximate Gradient by **mini batches** $m \ll N$

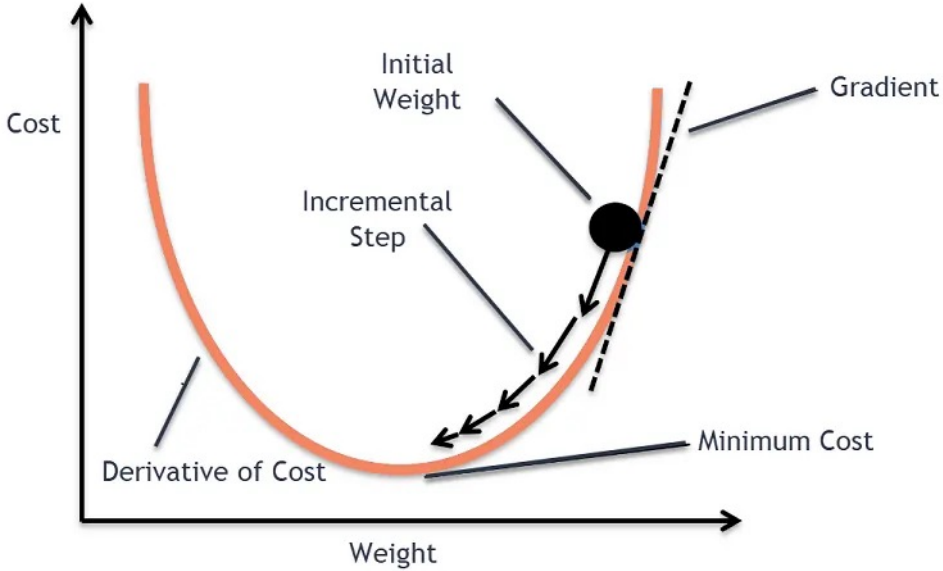
Finally: A small step into direction of $-\nabla C$,
i.e. adjust the weights and biases according to $-\nabla C$.

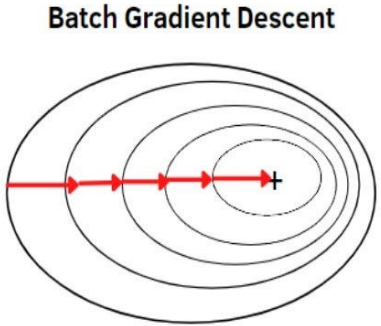
(Sample, Label)

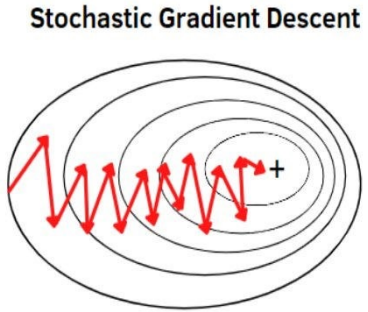


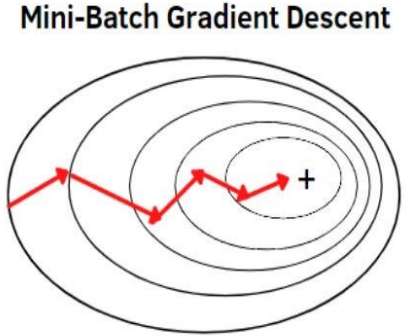
How does Neural Network learn?





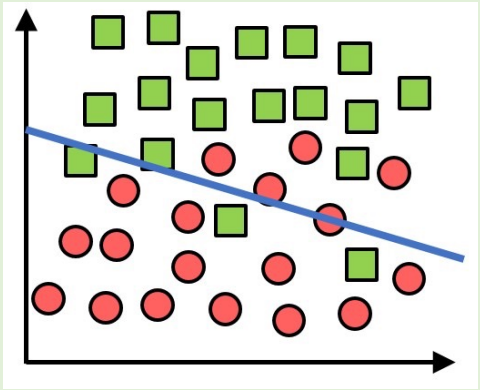






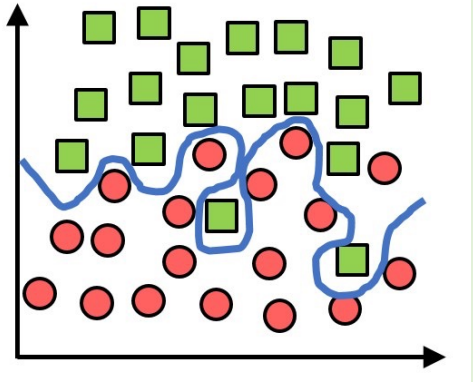
One can use other gradient descent optimization such as Adam, RMSprop, Momentum, etc

Underfitting & Overfitting



underfitting

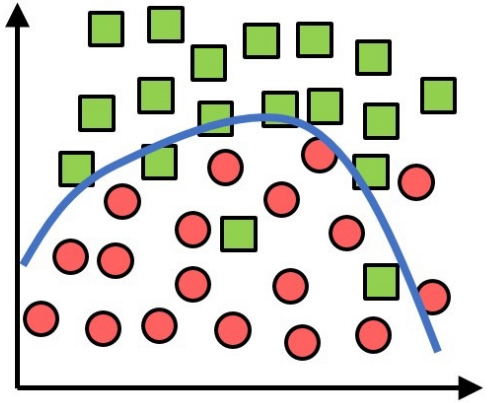
model is too simple to capture the underlying patterns in the training data, leading to poor performance on both training and new, unseen data



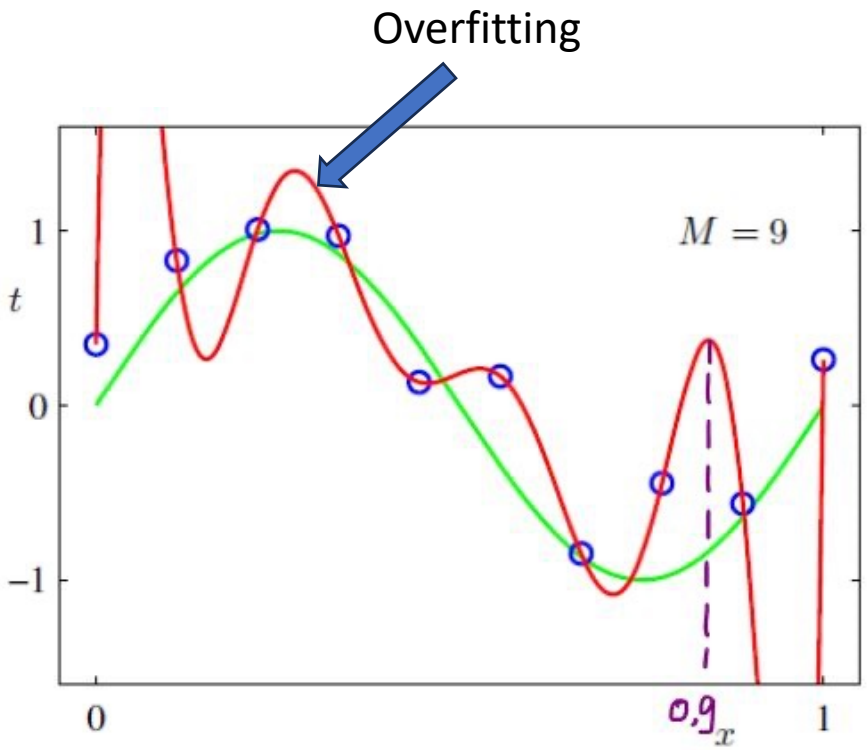
overfitting

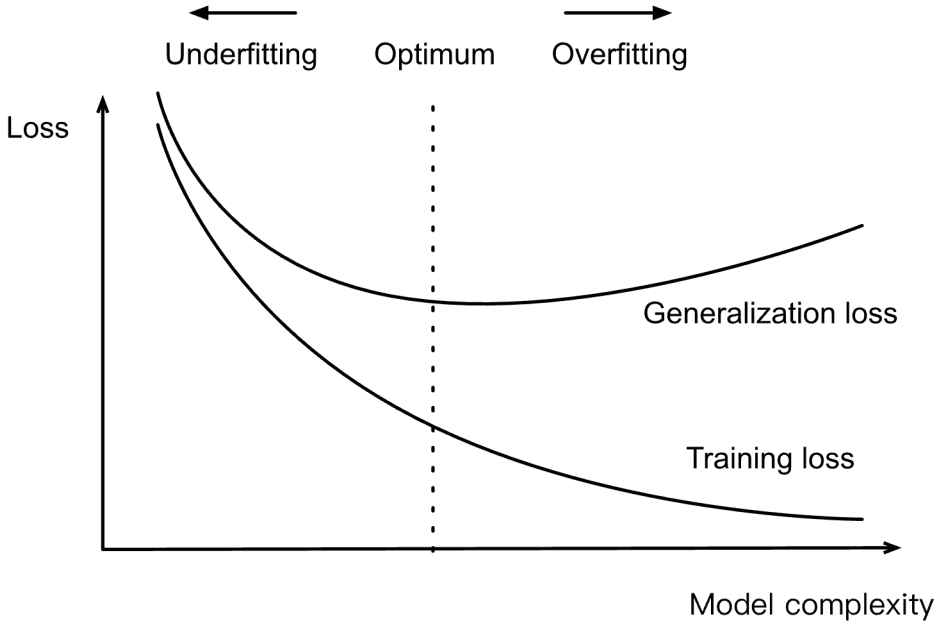
model learns its training data too closely, including noise, which causes it to perform poorly on new, unseen data

Optimal

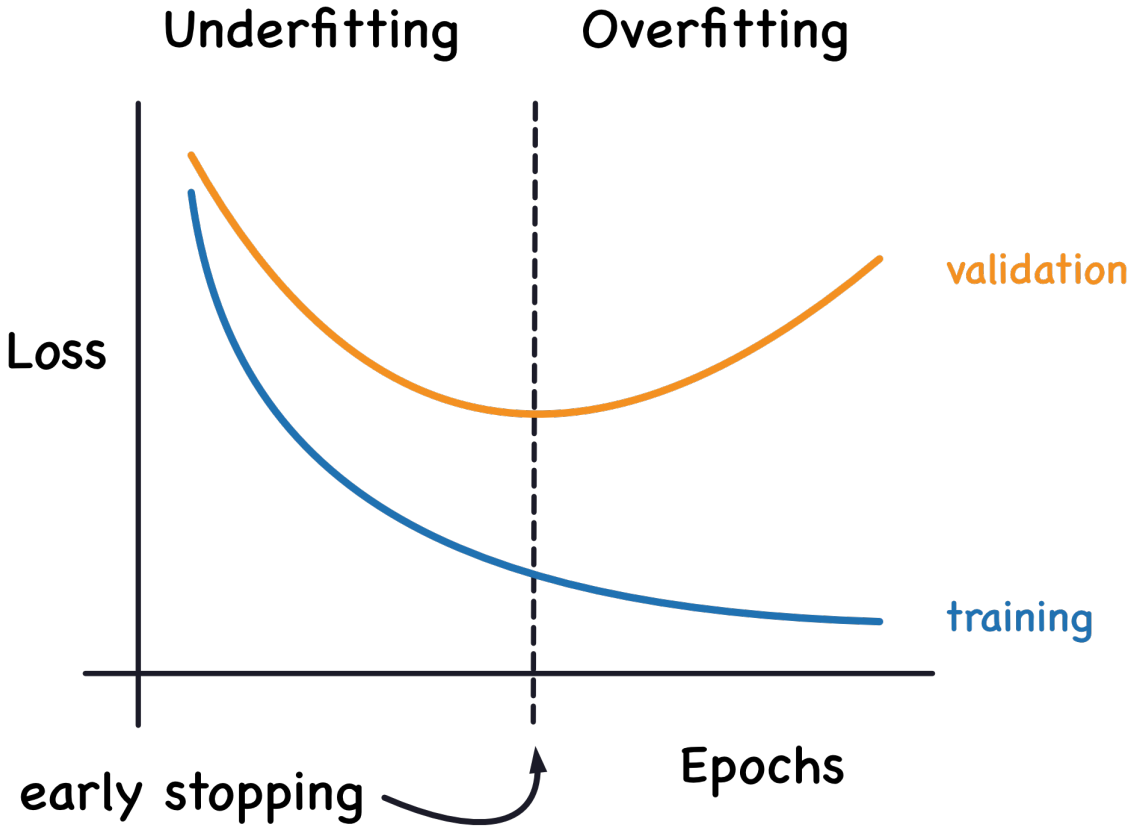


Underfitting & Overfitting





How to “avoid” overfitting?



How to “avoid” overfitting?

Regularization techniques to **prevent overfitting** by discouraging overly large weights

L1 Regularization

$$L_{\text{total}} = L_{\text{data}} + \lambda \sum_i |w_i|$$

Tends to produce sparse weights, meaning that many of the weights become exactly zero

L2 Regularization

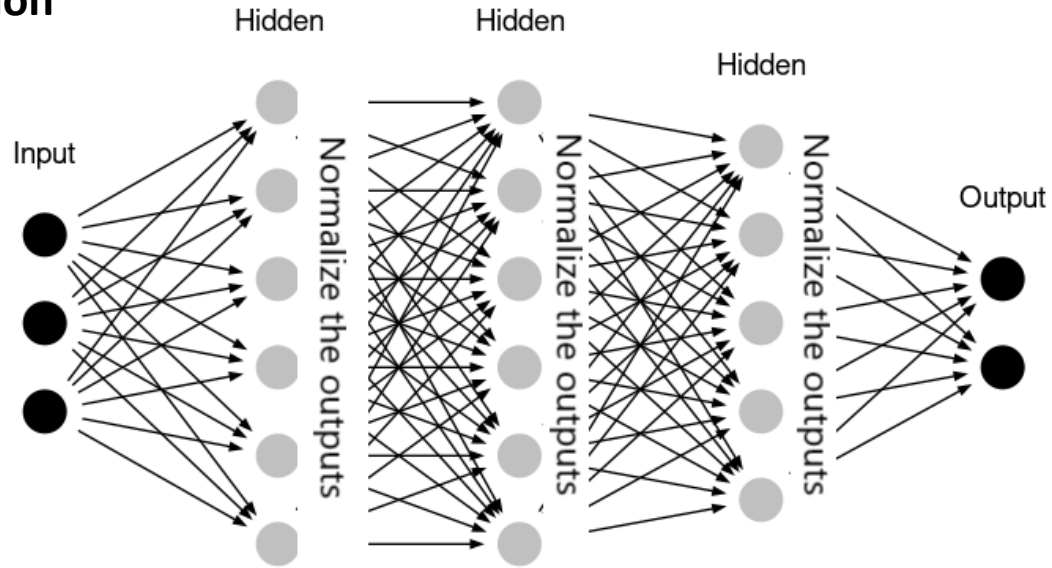
$$L_{\text{total}} = L_{\text{data}} + \lambda \sum_i w_i^2$$

Prefers to keep all weights small but not exactly zero

How to “avoid” overfitting?

a normalization technique used to make training faster and more stable by adjusting the inputs to each layer—re-centering them around zero and re-scaling them to a standard size

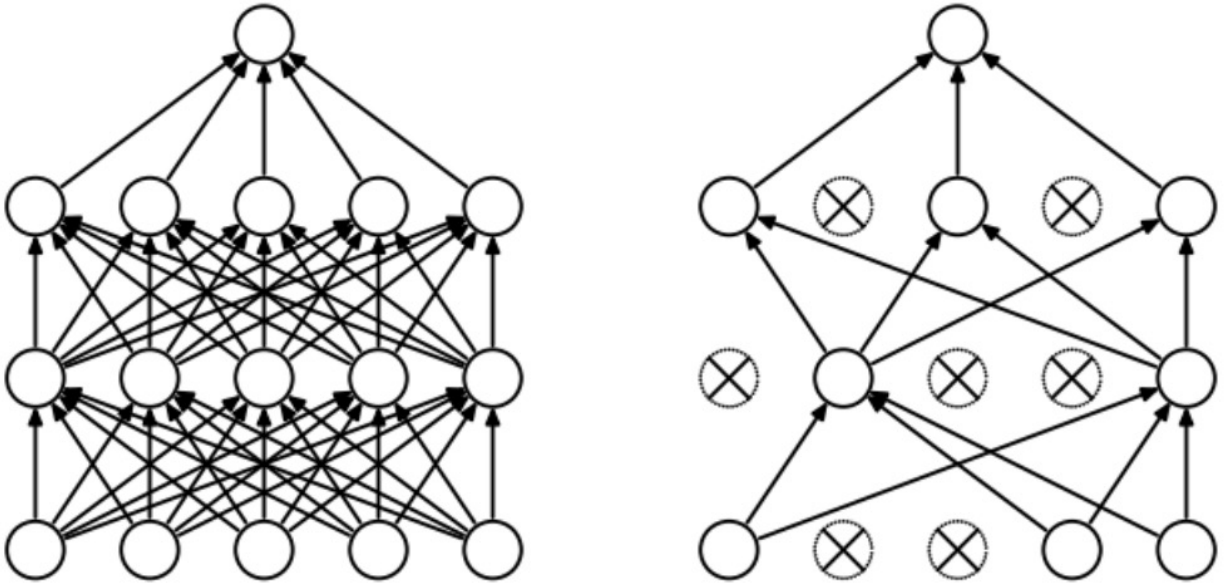
Batch Normalization



Batch Normalization allows us to use much higher learning rates and be less careful about initialization

How to “avoid” overfitting?

Dropout A regularization technique that randomly set the outputs of hidden neurons to zero. It is usually performed during the training process of a neural network



(a) Standard Neural Net

(b) After applying dropout.



Fashion MNIST Dataset

Label	Description	Examples
0	T-Shirt/Top	
1	Trouser	
2	Pullover	
3	Dress	
4	Coat	
5	Sandals	
6	Shirt	
7	Sneaker	
8	Bag	
9	Ankle boots	

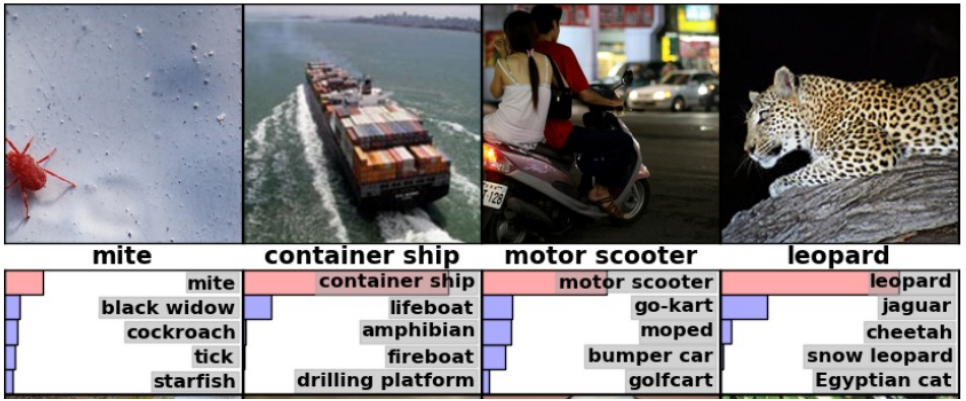
Beyond MNIST: The ImageNet task

ImageNet (by Feifei Li, 2010+)

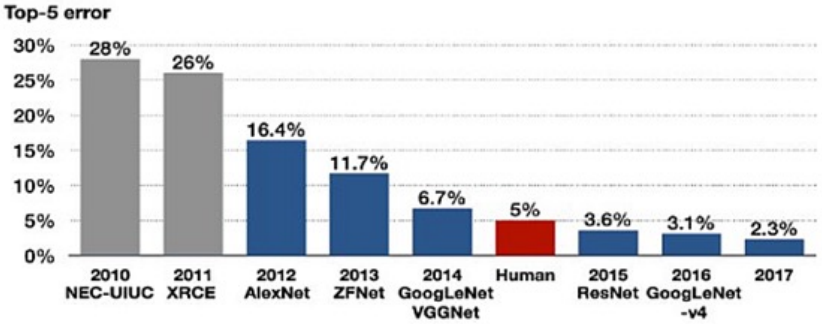


21,000 different categories

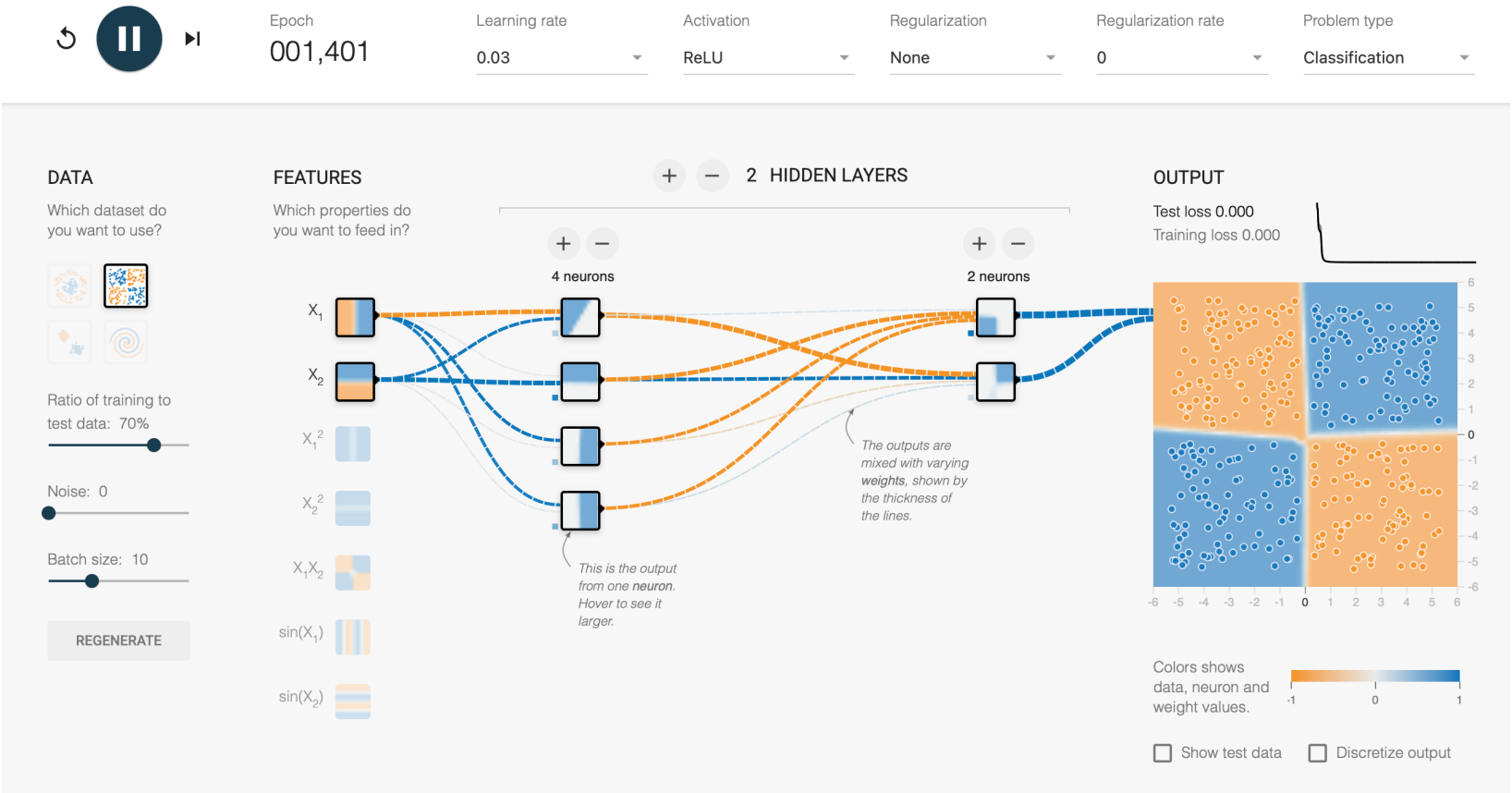
500 picture per category



Alex Krizhevsky, Ilya Sutskever, Geoffrey E. Hinton, NeurIPS 2012



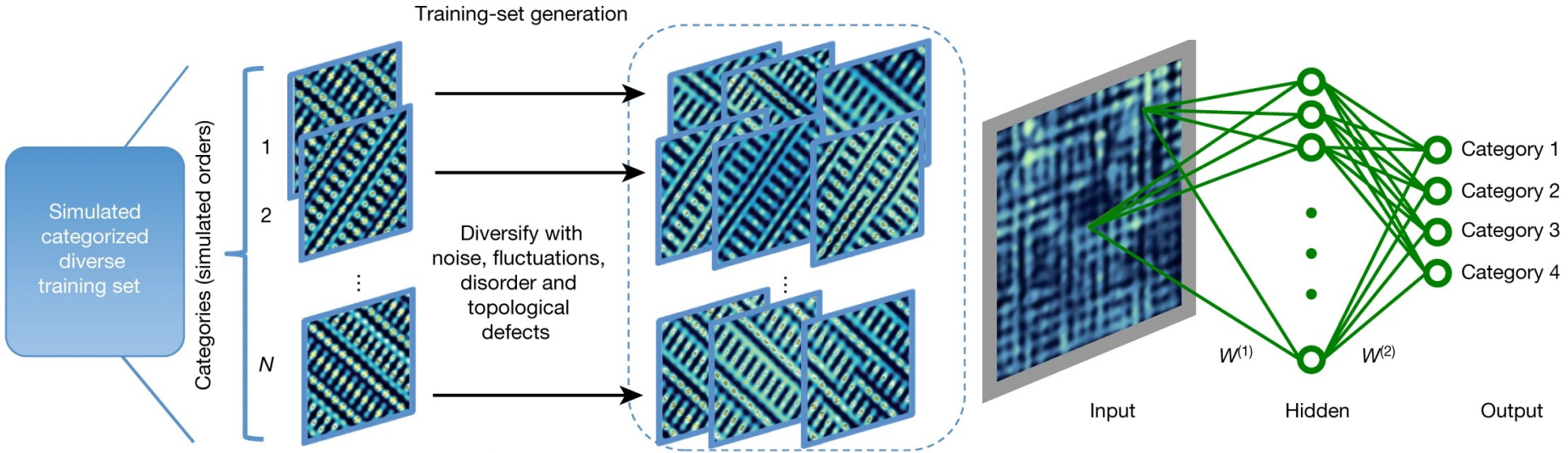
http://playground.tensorflow.org/

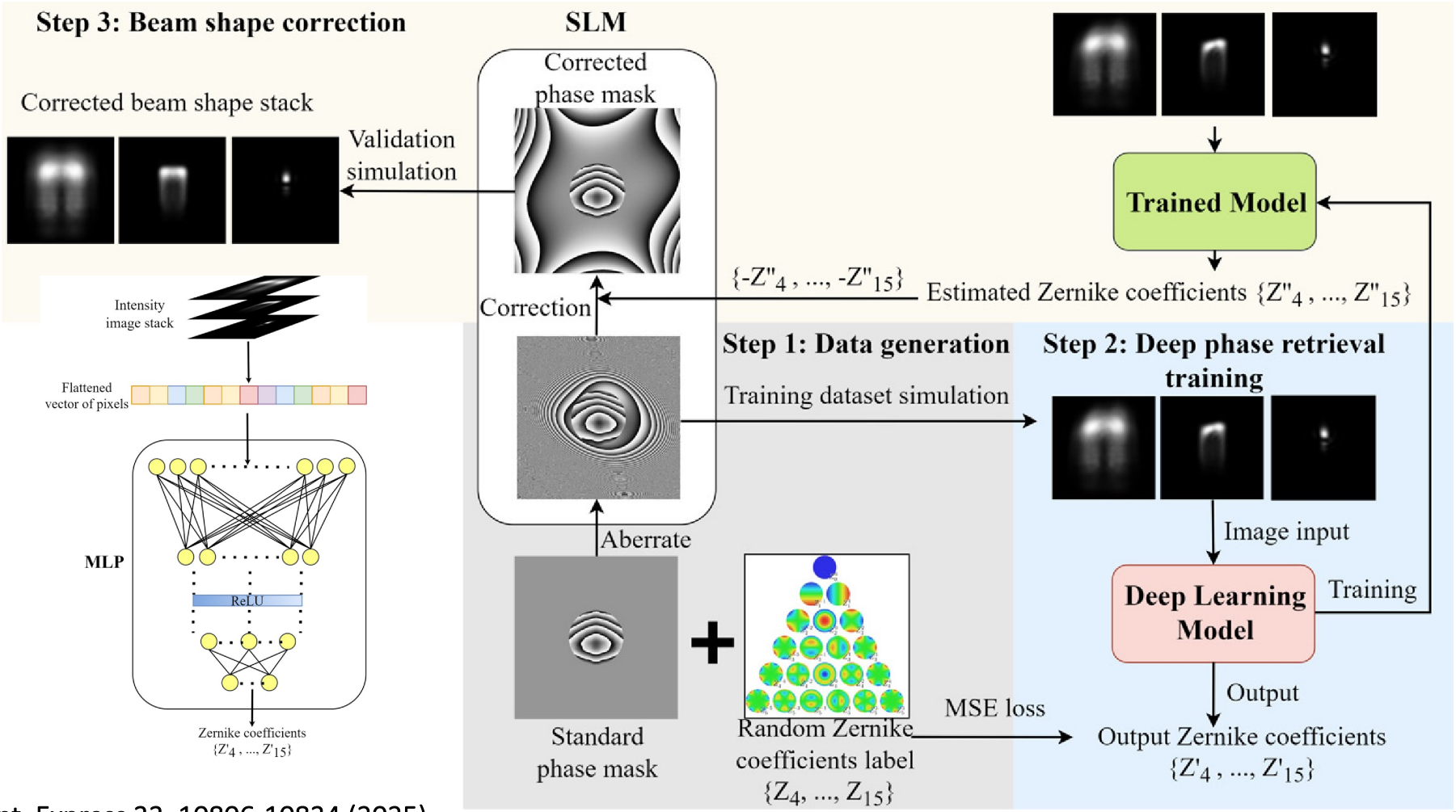


Examples

Nature 570, 484–490 (2019)

classification of experimental images from scanning tunneling microscopy of high-temperature superconductors

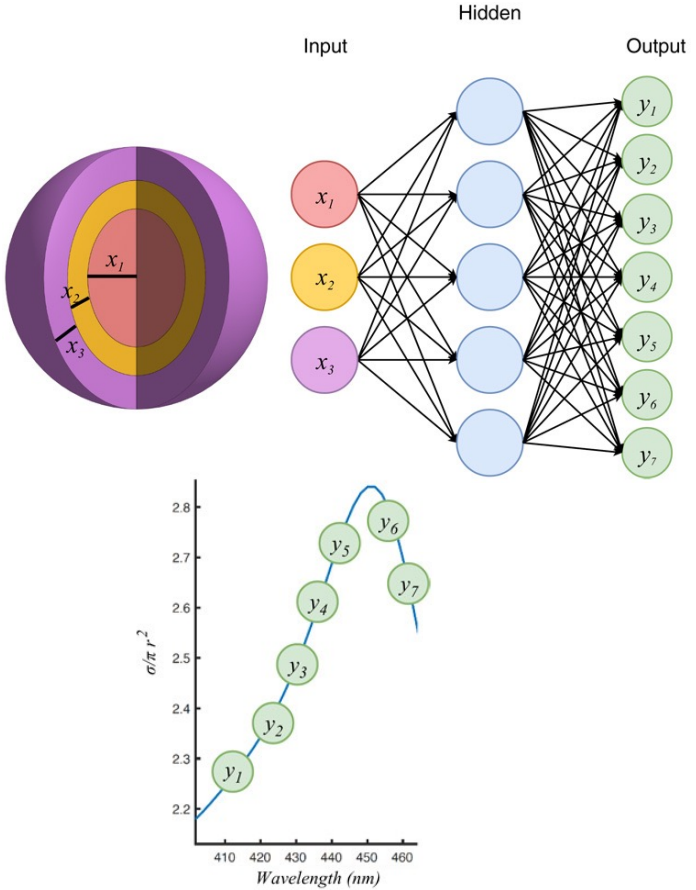


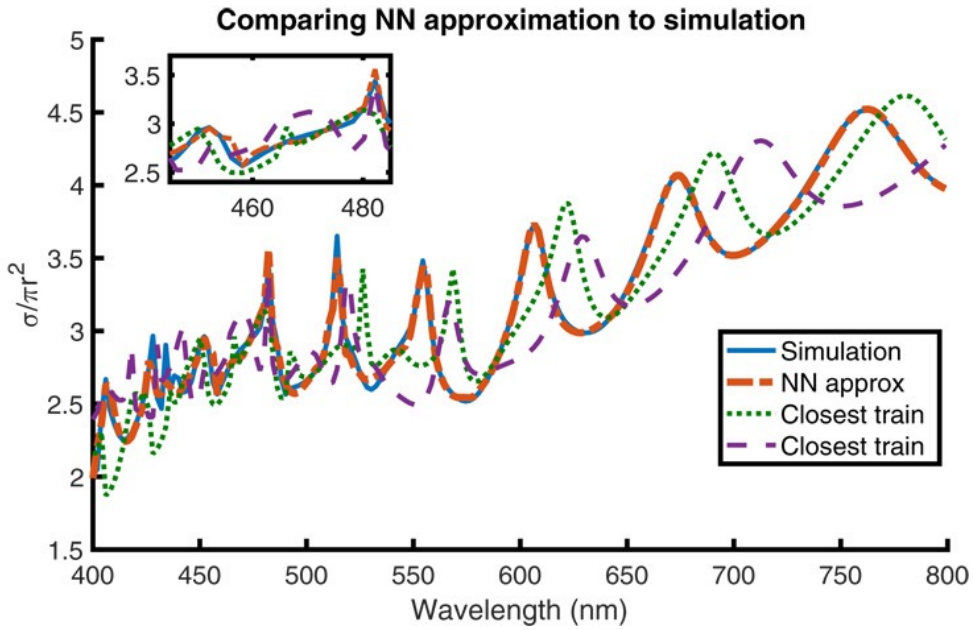


Opt. Express 33, 10806-10834 (2025)

Nanophotonic particle simulation

Sci. Adv. 4, eaar4206(2018)

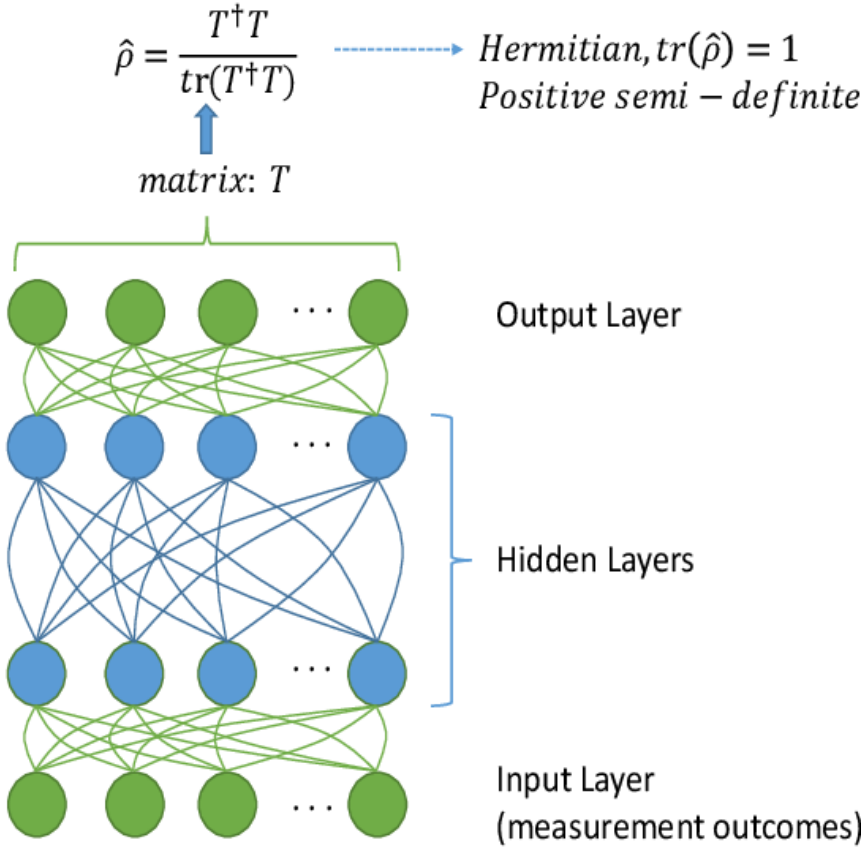




Quantum State Tomography

Reconstruct unknown quantum state from measurements

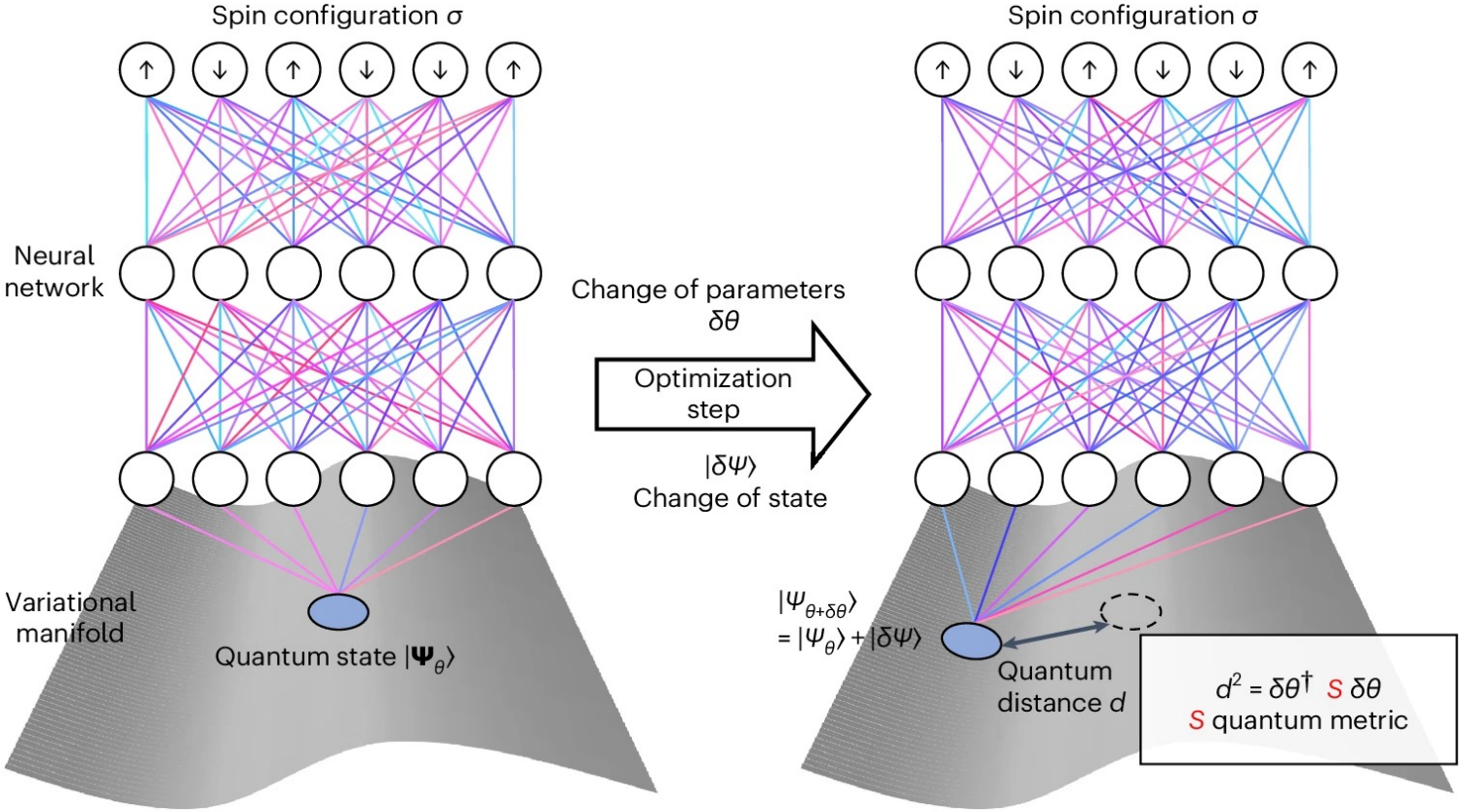
arXiv:1811.06654



Neural Quantum States

Nat. Phys. 20, 1476–1481 (2024)
Science 355,602-606(2017)

Representation of quantum states by neural networks





Is more data always better?

Does bigger always mean better in deep learning?

If NNs can approximate any function, does that mean they can solve any problem?

Are neural networks “black boxes”? Do we just not try hard enough?

…